Black-Box Configuration Search for Generalized Disjunctive Programming

Albert Joon Lee [1], Woosik Kim [2], and David E. Bernal Neira [1,*]

[1]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

[2]Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA

*Email: dbernaln@purdue.edu

Abstract

Selecting the configuration of a generalized disjunctive programming (GDP) model can require solving one nonlinear programming (NLP) subproblem for every candidate configuration. We test whether black-box search over an existing external integer representation of the logical decisions finds high-quality configurations with fewer subproblem solves than conventional GDP methods and discrete steepest descent (D-SDA). On four process GDP families (nine models) and 21 Grid test functions, we compare the surrogate methods RBFOpt, MISO, Branch-and-Model (BAM), and factorization-machine-based optimization with simulated annealing (FMQA-SA), and the direct-search method NOMAD, with D-SDA, uniform sampling, logic-based GDPopt methods, direct Big-M and Hull reformulations, and full enumeration, using common initial configurations, evaluation budgets, and stored subproblem responses. On continuous stirred-tank reactor (CSTR) networks, the radial-basis-function methods RBFOpt and MISO reach the reference objective in 45 and 43 of 60 runs, compared with 21 for NOMAD and 16 for D-SDA, whose descents mostly end at the same local minimum with or without restarts. On the largest network, MISO and RBFOpt reach the reference in ten and nine of ten runs with medians of 29.5 and 31.5 NLP solves, whereas the logic-based methods solve up to 455 subproblems, nearly one for each admissible configuration, and miss it. On the two smallest design models, exact paired tests detect no difference between D-SDA and RBFOpt, MISO reaches all ten Column targets, and a direct reformulation solves the smallest in half a second. On the Grid functions, MISO and RBFOpt reach 86.2% and 81.4% of the targets at a budget of 100 evaluations, compared with 60.0% for NOMAD and 48.1% for D-SDA. MISO reaches the most targets overall but spends more search time than RBFOpt on several conditions. On a pharmaceutical flowsheet whose configurations are evaluated by simulation-based optimization in PharmaPy, where GDP solvers do not apply, RBFOpt and MISO find the best of 36 configurations in every run with 20 evaluations. Black-box configuration search is most useful when the configuration space is too large to enumerate and contains deceptive local minima, or when configurations are evaluated by simulation; for small algebraic models, enumeration or direct reformulation is simpler.

Keywords: Generalized disjunctive programming; Black-box optimization; Derivative-free optimization; Surrogate modeling; Process systems engineering; Computational benchmarking.

1 Introduction

In chemical process design, evaluating a single candidate configuration can require costly optimization or simulation, making the choice of which configuration to explore next a central computational challenge. Process design involves discrete decisions, such as equipment selection, the number of processing stages, and operating modes, along with continuous variables such as flow rates, temperatures, and equipment sizes. Generalized disjunctive programming (GDP) provides a framework to represent these decisions through

Boolean variables, logical propositions, and conditional constraints (Raman and Grossmann, 1994). In logic-based GDP solution methods, fixing the Boolean variables associated with a selected configuration determines which constraints remain active and yields a reduced optimization subproblem. When all discrete decisions are fixed, this subproblem can be formulated as a nonlinear programming (NLP) problem. Exploring multiple configurations may therefore require repeatedly solving NLP subproblems, which can be computationally expensive. Consequently, identifying high-quality configurations with fewer evaluations of NLP subproblems is an important objective in solving GDP problems.

Conventional GDP solution methods generally rely on mixed-integer nonlinear programming (MINLP) reformulations or logic-based decomposition. The Big-M and Hull reformulations transform disjunctive constraints into algebraic formulations (Grossmann and Lee, 2003; Raman and Grossmann, 1994), while logic-based approaches select discrete configurations and solve the corresponding reduced subproblems (Bergamini et al., 2005; Türkay and Grossmann, 1996). In logic-based approaches, the strategy used to select discrete configurations can significantly affect the number of subproblem evaluations required.

Discrete-Steepest Descent (D-SDA) was previously applied to process superstructure optimization by Liñán et al. (2020). Building on this approach, our previous work introduced the Logic-Based Discrete-Steepest Descent (LD-SDA), which represents ordered Boolean decisions using external integer variables (Ovalle et al., 2025). LD-SDA explores the resulting integer space through neighborhood search and discrete line search, evaluating selected configurations by solving their corresponding reduced optimization subproblems. However, as a local search method, LD-SDA may converge to different locally optimal solutions depending on the starting point and the selected neighborhood (Peng et al., 2024).

We use the existing external integer representation to search GDP configurations with black-box optimization methods. Surrogate-based derivative-free optimization (DFO) uses accumulated objective observations to guide new evaluations (Larson et al., 2019). RBFOpt builds radial basis function surrogates and balances predicted objective values with exploration (Costa and Nannicini, 2018), and MISO combines radial basis function surrogates with sampling strategies designed for integer variables (Müller, 2016). The comparison also includes Branch-and-Model (BAM) (Ma et al., 2023) and a variant of the factorization machine with quantum annealing (FMQA) framework (Kitai et al., 2020), in which classical simulated annealing replaces quantum annealing; we denote this variant FMQA-SA. FMQA represents its surrogate as a quadratic unconstrained binary optimization problem, the input format of quantum annealers, which makes it a candidate for quantum-assisted configuration search. We also include the mesh adaptive direct-search solver NOMAD (Audet et al., 2022), which, with MISO, performed best on mixed-integer problems in the comparison of Ploskas and Sahinidis (2022). We ask whether these methods find high-quality configurations with fewer NLP subproblem solves than D-SDA and conventional GDP methods, and when this advantage translates into shorter computing time.

Candidate selection has its own cost, and evaluation costs vary between configurations (Moré and Wild, 2009). We therefore report three measures for every method: how often it reaches the reference objective, how many NLP subproblems it solves, and its estimated completion time. Full enumeration of each configuration domain and uniform sampling serve as references, and all methods share stored configuration responses, initial configurations, and evaluation budgets. The contributions are as follows:

- A comparison of surrogate-based and direct-search configuration methods with local descent, logic-based and direct GDP methods, and full enumeration on four process GDP families and 21 Grid test functions, measured by target attainment, NLP subproblems solved, and completion time.
- Evidence from execution records that the logic-based methods solve one subproblem for each admissible configuration of these models, whereas surrogate search reaches near-optimal configurations with a small fraction of those subproblems.
- An extension of the external-variable formulation to subproblems defined by a process simulator, demonstrated on a lomustine manufacturing flowsheet simulated in PharmaPy.
- An analysis of the landscape features that determine when surrogate search outperforms local descent, with recommendations for choosing among enumeration, direct reformulation, local descent, and surrogate search.

2 Background

2.1 Generalized Disjunctive Programming and Fixed-Configuration Evaluation

GDP represents operational and design alternatives using Boolean variables and associated algebraic constraints (Raman and Grossmann, 1994). The original continuous variables have domain $\mathcal{X} = [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \subseteq \mathbb{R}^{n_x}$. The original integer variables have domain $\mathcal{Z} = [\underline{\mathbf{z}}, \overline{\mathbf{z}}] \cap \mathbb{Z}^{n_z}$. Let K index the disjunctions and D_k index the disjuncts in disjunction $k \in K$. Each disjunct $i \in D_k$ has a Boolean variable Y_{ik}. The GDP formulation is given by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{z},\mathbf{Y}} \quad & f(\mathbf{x},\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\ & \veebar_{i \in D_k} \begin{bmatrix} Y_{ik} \\ \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \end{bmatrix}, \qquad k \in K, \\ & \Omega(\mathbf{Y}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}, \quad \mathbf{Y} \in \{\text{False}, \text{True}\}^{n_y}. \end{aligned} \tag{GDP}$$

Here, f is the objective function, $\mathbf{g}$ represents the global constraints, and $\mathbf{h}_{ik}$ represents the constraints associated with each disjunct. The operator $\veebar$ requires that exactly one disjunct be selected within each disjunction. Logical propositions $\Omega(\mathbf{Y})$ impose additional relationships between Boolean variables.

Following the external-variable reformulation of Ovalle et al. (2025), each ordered group of m_j Boolean alternatives is represented by an external integer variable $\xi_j \in \{1, \dots, m_j\}$, which identifies the selected alternative. Ordered alternatives, such as the position of a feed along a column, are one of the conditions that Ovalle et al. (2025) use because they make neighboring values of ξ_j physically similar configurations. The representation itself does not require this order: unordered alternatives, such as a choice among equipment types, can be indexed in any fixed order, as in Section 3.2. Let n_E denote the number of external integer variables, and $\mathcal{Y}_{\text{logic}}$ the set of Boolean assignments that satisfy the conditions `exactly-one`, and $\Omega(\mathbf{Y})$. The external-variable lattice and its logically admissible subset are

$$\begin{aligned} \mathcal{L}_E &= \prod_{j=1}^{n_E} \{1, \dots, m_j\}, \\ \mathcal{E}_{\text{adm}} &= \{\boldsymbol{\xi} \in \mathcal{L}_E : \mathbf{Y}(\boldsymbol{\xi}) \in \mathcal{Y}_{\text{logic}}\}. \end{aligned} \tag{1}$$

For each logically admissible configuration $\boldsymbol{\xi}$, the corresponding Boolean variables are fixed, yielding the following reduced optimization subproblem:

$$\begin{aligned} \Phi(\boldsymbol{\xi}) = \inf_{\mathbf{x},\mathbf{z}} \quad & f(\mathbf{x},\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\ & \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \quad \text{for all } (i,k) \text{ with } Y_{ik}(\boldsymbol{\xi}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}. \end{aligned} \tag{Sub}$$

Here, $\Phi(\boldsymbol{\xi})$ represents the optimal objective value of the reduced subproblem. The reduced subproblem is an NLP when all remaining decision variables are continuous. If the reduced subproblem is infeasible, we define $\Phi(\boldsymbol{\xi}) = +\infty$.

When the external-variable mapping represents all logically admissible Boolean configurations, the original GDP can be expressed as the following upper-level optimization problem (Ovalle et al., 2025):

$$\min_{\boldsymbol{\xi} \in \mathcal{E}_{\text{adm}}} \Phi(\boldsymbol{\xi}). \tag{2}$$

A logically admissible configuration may still yield an infeasible reduced subproblem.

Problem (2) uses only the value $\Phi(\boldsymbol{\xi})$ of each configuration, so it also applies when a configuration is evaluated by a process simulator. Fixing $\boldsymbol{\xi}$ then selects the unit models of a flowsheet, and the simulator

$\mathcal{S}_{\boldsymbol{\xi}}$ computes its states $\mathbf{s}$, such as stream compositions and crystal size distributions, from operating inputs $\mathbf{u} \in \mathcal{U}_{\boldsymbol{\xi}}$. The reduced subproblem becomes

$$\Phi(\boldsymbol{\xi}) = \min_{\mathbf{u} \in \mathcal{U}_{\boldsymbol{\xi}}} \big\{ f(\mathbf{u}, \mathbf{s}) : \ \mathbf{s} = \mathcal{S}_{\boldsymbol{\xi}}(\mathbf{u}), \ \mathbf{g}(\mathbf{u}, \mathbf{s}) \leq \mathbf{0} \big\}, \tag{Sim}$$

whose objective and constraints are known only through simulation outputs. A derivative-free method in $\mathbf{u}$ solves Problem (Sim), so $\Phi(\boldsymbol{\xi})$ is approximated by the best value that this inner search finds within its simulation budget. Logic-based and direct GDP methods require the equations of every disjunct and do not apply to Problem (Sim). The configuration-search methods compared here use it through the same evaluation interface as an NLP subproblem, which Section 3.2 illustrates with a pharmaceutical flowsheet simulated in PharmaPy (Casas-Orozco et al., 2021).

2.2 Conventional GDP Solution Approaches

Big-M and Hull reformulations transform GDP models into mixed-integer nonlinear programming (MINLP) formulations by associating Boolean variables Y_{ik} with binary variables y_{ik} and translating logical propositions into algebraic constraints (Grossmann and Lee, 2003; Raman and Grossmann, 1994). In the Big-M reformulation, the disjunctive constraints are expressed as $\mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{M}_{ik}(1 - y_{ik})$, where the inequality is applied componentwise. The Big-M values must be sufficiently large to render the corresponding constraints redundant when $y_{ik} = 0$, but excessively large values can weaken the continuous relaxation and cause numerical difficulties.

Hull reformulations introduce disaggregated variables, linking constraints, and bounds scaled by binary indicators (Grossmann and Lee, 2003). Disjunctive constraints are imposed on the disaggregated variables when $y_{ik} = 1$, while $y_{ik} = 0$ forces these variables to zero, making the corresponding disjunct inactive. For convex disjunctive sets, this formulation can provide a stronger continuous relaxation than Big-M, although it introduces additional variables and constraints. For nonconvex GDP problems, the Hull reformulation does not guarantee a convex reformulated model or global optimality (Chen et al., 2022).

Logic-based methods address GDP problems directly by exploiting their logical and disjunctive structure to select discrete configurations and construct the corresponding optimization subproblems. Logic-based outer approximation (LOA) alternates between a mixed-integer master problem and fixed-configuration NLP subproblems (Türkay and Grossmann, 1996). Under suitable convexity and regularity assumptions, the master problem uses linearization cuts to obtain lower bounds, while feasible solutions to the NLP subproblem provide upper bounds. For nonconvex GDP problems, these linearizations may not provide valid global lower bounds, motivating global logic-based outer approximation (GLOA), which requires globally valid relaxations and subproblem bounds. Bergamini et al. (2005) developed a GLOA approach for process networks using piecewise under- and overestimators.

GDPopt provides implementations of logic-based methods within Pyomo.GDP (Chen et al., 2022; Pyomo Development Team, n.d.). Its relaxation with integer cuts (RIC) method uses a mixed-integer linear programming (MILP) relaxation to select configurations and adds no-good cuts to exclude evaluated discrete assignments, without generating the successive outer-approximation cuts used by LOA (Pyomo Development Team, n.d.).

LD-SDA is an existing alternative that searches ordered external configurations by improving neighborhood moves and discrete line searches (Ovalle et al., 2025). It selects the best improving neighbor, continues along the resulting direction while improvement persists, and resumes neighborhood exploration. The present work uses this discrete-search structure as a reference and compares other configuration-selection strategies and their computational costs.

2.3 Derivative-Free Optimization Solvers

Derivative-free optimization (DFO) uses function observations without requiring derivatives of the objective with respect to the search variables (Larson et al., 2019). Black-box configuration search selects configurations using their evaluation responses without accessing the subproblem equations. For GDP configuration search, the outer method can be derivative-free, even if its evaluation routine uses a derivative-based nonlinear solver. D-SDA also uses this derivative-free interface, although it does not construct a surrogate model.

The upper-level search operates over external integer variables that represent the logical decisions in the GDP. Some logical restrictions are embedded in the external-variable reformulation, while additional constraints can be imposed on the configuration space. One approach to DFO uses surrogate models to guide the search. Surrogate methods seek to reduce expensive evaluations by fitting models to observed data, with computational cost arising from both objective evaluations and model-based candidate selection.

Several DFO methods handle integer or categorical variables directly, and Ploskas and Sahinidis (2022) compare many of them on mixed-integer test problems. Mesh adaptive direct search, implemented in `NOMAD`, extends pattern search to mixed variables through user-defined discrete neighborhoods (Abramson et al., 2009; Audet et al., 2022), and `MISO` combines radial basis function surrogates with sampling strategies for integer variables (Müller, 2016). Bayesian optimization has also been adapted to combinatorial spaces, for example by fitting sparse second-order models of binary variables in BOCS (Baptista and Poloczek, 2018), which are closely related to the factorization-machine surrogate used here, or by placing kernels on graphs of discrete configurations in COMBO (Oh et al., 2019). We compare four surrogate-based methods that represent different modeling choices: `RBFOpt` and `MISO` interpolate all observations with global radial basis function models but select candidates differently, `BAM` partitions the domain and fits local algebraic models, and `FMQA-SA` fits a quadratic model over a binary encoding. `NOMAD` adds a direct-search method that polls trial configurations on an adaptive mesh and keeps no global surrogate.

2.3.1 Branch-and-Model (BAM)

Branch-and-Model (BAM) combines domain partitioning with local surrogate construction (Ma et al., 2023). The evaluated points define boxes, and a potentially optimal selection procedure identifies regions for further modeling and search. Within selected boxes, Automated Learning of Algebraic Models (ALAMO) constructs sparse algebraic models from nearby observations (Cozad et al., 2014). BARON optimizes these local models to propose candidates, while box-size exploration maintains broader coverage (Ma et al., 2023; Sahinidis, 1996; The Optimization Firm, 2026). Within BAM, BARON optimizes these learned models and never receives the GDP itself. Its support for integer restrictions allows for the configuration domains considered here (The Optimization Firm, 2026).

2.3.2 Factorization-Machine-Based Optimization

A second-order factorization machine (FM) models individual feature effects and pairwise interactions through shared latent parameters (Rendle, 2010). For a binary encoding $\mathbf{b} \in \{0,1\}^m$ of an external integer configuration $\boldsymbol{\xi}$, the FM fit surrogate is

$$\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b}) = \beta_0 + \sum_{i=1}^{m} \beta_i b_i + \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} \langle \mathbf{a}_i, \mathbf{a}_j \rangle b_i b_j, \tag{3}$$

where $\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b})$ denotes the predicted training response, m is the number of binary features, β_0 is the intercept, β_i is the linear coefficient of b_i, and $\mathbf{a}_i \in \mathbb{R}^r$ is its latent vector of dimension r. The inner product $\langle \mathbf{a}_i, \mathbf{a}_j \rangle$ determines the pairwise interaction coefficient. The model parameters are learned from the evaluated configurations and their corresponding training responses.

For binary inputs, $b_i^2 = b_i$, the fitted model is a binary quadratic model. Excluding the constant term β_0 gives the following quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{b} \in \{0,1\}^m} \mathbf{b}^{\mathsf{T}} \mathbf{Q} \mathbf{b}, \qquad Q_{ij} = \begin{cases} \beta_i, & i = j, \\ \langle \mathbf{a}_i, \mathbf{a}_j \rangle, & i < j, \\ 0, & i > j. \end{cases} \tag{4}$$

The QUBO models the learned surrogate, and a binary string may decode to a configuration outside the domain or one that violates the logical constraints.

The factorization machine with quantum annealing (FMQA) framework combines FM surrogate fitting with annealing-based candidate generation (Kitai et al., 2020). Based on observed evaluations, the method fits the FM, samples binary candidates from the resulting quadratic model, and evaluates a selected decoded configuration through the objective-evaluation routine before updating the training data.

The FMQA simulated annealing variant (FMQA-SA) uses classical simulated annealing to sample binary candidates from the learned QUBO. Temperature-dependent acceptance of uphill moves supports exploration, while gradual cooling favors lower-energy solutions (Kirkpatrick et al., 1983). In this study, no quantum hardware was used.

2.3.3 RBFOpt

RBFOpt is an open-source surrogate optimizer for bounded black-box problems, with support for continuous, integer, and categorical variables (Costa and Nannicini, 2018; Nannicini, 2021). Given N observed objective values in external integer configurations $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(N)}$, RBFOpt constructs a radial basis function (RBF) surrogate model of the form

$$\widehat{\Phi}_N(\boldsymbol{\xi}) = \sum_{r=1}^{N} \lambda_r \varphi\Big(\|\boldsymbol{\xi} - \boldsymbol{\xi}^{(r)}\|_2 \Big) + p(\boldsymbol{\xi}), \tag{5}$$

where $\widehat{\Phi}_N(\boldsymbol{\xi})$ approximates the objective value for a candidate configuration $\boldsymbol{\xi}$, λ_r are interpolation coefficients determined from the observed values, φ is the RBF kernel evaluated at the Euclidean distance between configurations, and $p(\boldsymbol{\xi})$ is a polynomial tail depending on the selected kernel.

Candidate selection balances low predicted values against exploration away from evaluated points. The metric stochastic response surface method (MSRSM) combines scaled surrogate predictions and distances to existing samples, changing their relative weights during the search (Regis and Shoemaker, 2007). New objective evaluations update the model.

2.3.4 MISO

MISO uses the radial basis function surrogate of Equation (5) but selects candidates with strategies designed for integer and mixed-integer variables (Müller, 2016). Its CPTV strategy first perturbs randomly chosen coordinates of the best configuration found and scores the candidates by surrogate value and distance to evaluated points, as MSRSM does. When this stage stops improving, it switches to a target-value step, which proposes the configuration at which the surrogate could attain a prescribed target value with the least added bumpiness.

2.3.5 NOMAD

NOMAD implements mesh adaptive direct search (MADS) (Audet et al., 2022). Each poll evaluates trial points on a mesh around the incumbent, and the mesh is refined after unsuccessful polls and coarsened after successful ones; for integer variables, it cannot become finer than the integer spacing (Abramson et al., 2009). By default, NOMAD 4 complements the poll with search steps based on local quadratic models, but it keeps no global surrogate of the objective.

3 Experimental Methods

3.1 Process GDP benchmark problems

The process benchmarks include a CSTR network (Liñán et al., 2020), a 17-stage distillation column (Ghouse et al., 2018), and a small batch plant (Kocis and Grossmann, 1988). We use their GDP formulations reported by Ovalle et al. (2025), and the nine-stage nonlinear mode-switching model of Peng et al. (2024). Their objectives minimize the total reactor volume (`CSTR`), condenser and reboiler duties, plus the active tray cost (`Column`), the investment cost (`Small Batch`), and the negative integral of the squared state (`Dynamic`). The external variables represent the reactor/feed cutoff and recycle location (`CSTR`), reflux and boilup positions (`Column`), parallel unit counts (`Small Batch`), and final stages of modes 1 and 2 (`Dynamic`). For these process benchmarks, fixing the external choices leaves only continuous variables, so the resulting subproblems are NLPs. The `CSTR` superstructure contains N potential reactor positions, where $N \in \{5, 10, 15, 20, 25, 30\}$. We denote each `CSTR` instance by its size parameter N, such as `CSTR30` for $N = 30$. `Dynamic` uses 30 finite elements per stage and three Radau collocation points. The reference configurations in external variable

coordinates are (N, N) for `CSTR`, $(13, 4)$ for `Column` (physical trays $(14, 5)$), $(2, 2, 1)$ for `Small Batch`, and $(1, 4)$ for `Dynamic`. The full GDP formulations of the `CSTR`, `Column`, and `Small Batch` benchmarks are provided in Appendices A.1-A.3 of Ovalle et al. (2025), respectively, while the `Dynamic` formulation is described in Section IV.C of Peng et al. (2024). External-variable mappings for the three design cases are provided in Appendices B.1, B.2, and B.4 of Ovalle et al. (2025). The `CSTR` volume is reported in liters and the `Small Batch` investment in the source model's dollars. The `Column` uses the source model's weighted cost proxy based on duties and active trays. The model sizes and unit conventions are given in the Supplementary Section S4 (Table S3). `Dynamic` is an abstract benchmark with no calibrated physical units for state or time. Minimizing its negative squared-state integral favors greater integrated state magnitude and tests mode sequencing coupled to dynamic equations.

Table 1 summarizes the original GDP and DFO candidate domains, the number of configurations with `IPOPT`-accepted objective values, the number of configurations with measured evaluation times, and the evaluation budgets. The process searches use frozen objective tables from fixed-configuration `IPOPT` evaluations; only accepted finite objectives are used as valid lookup objectives. The objective, acceptance status, and point cost are taken from the same recorded evaluation attempt for the same model, CSTR size, and external-variable tuple. Measured coverage includes all configurations in each DFO candidate box, including unsuccessful and structural-rejection evaluations. The evaluation budget B limits the total number of objective evaluations, including initialization, repeated calls, and unsuccessful evaluations. The full GDP box precedes the logical and algebraic feasibility restrictions. `Column` searches $\{8, \ldots, 15\} \times \{1, \ldots, 8\}$ (64 coordinates), the logical domain of its original $\{1, \ldots, 15\}^2$ box. Other cases use their full boxes. The `Column` bounds encode the reflux and boilup placement restrictions; algebraic constraints are still checked during configuration evaluation. All DFO methods use this box, and the `Column` conclusions apply only to it. For `CSTR`, recycle/feed ordering admits $N(N+1)/2$ logical candidates, but not all yield objective values satisfying the solver acceptance criteria defined in Section 3.5. The `CSTR` comparisons do not isolate scalability: evaluation budgets change with size. `Dynamic` admits only configurations with $\xi_1 < \xi_2$ or $(\xi_1, \xi_2) = (9, 9)$, which leaves 37 logically admissible configurations. `Small Batch` uses all 27 configurations in $\{1, 2, 3\}^3$, whose decisions count parallel mixer, reactor, and centrifuge units. Of these configurations, 15 are rejected during feasibility-based bound tightening (FBBT) preprocessing.

Table 1: Process GDP domains, `IPOPT`-accepted lookup coverage, measured coordinates, evaluation budgets, and reference objective values. The DFO domain equals the full GDP box except for `Column`, which uses $\{8, \ldots, 15\} \times \{1, \ldots, 8\}$. Measured coordinates cover the complete DFO box in every case. Model sizes and unit conventions are given in Supplementary Table S3.

Model	N	Full GDP box	Full box size	Finite lookup	DFO size / measured	B	f_{ref}
`Column`	–	$\{1, \ldots, 15\}^2$	225	29	64	13	19346.1
`CSTR`	5	$\{1, \ldots, N\}^2$	25	15	25	8	3.06201
	10		100	54	100	15	2.88954
	15		225	120	225	22	2.82939
	20		400	210	400	29	2.79941
	25		625	325	625	36	2.78148
	30		900	464	900	43	2.76957
`Small Batch`	–	$\{1, 2, 3\}^3$	27	12	27	10	167428
`Dynamic`	–	$\{1, \ldots, 9\} \times \{2, \ldots, 9\}$	72	37	72	12,15,20, 30,37,50	-212.009

The 980 primary process runs (`D-SDA`, `NOMAD`, `RBFOpt`, `MISO`, `BAM`, `FMQA-SA`, and uniform sampling; 14 model-budget conditions and ten seeds per condition) visit 2,106 distinct model/size/coordinate keys.

3.2 Simulation-based benchmark: lomustine manufacturing

The `Lomustine` benchmark selects the flowsheet of a manufacturing process for the anticancer drug lomustine whose model is a PharmaPy simulation (Casas-Orozco et al., 2021, 2023). The superstructure was introduced by Laky and Casas-Orozco (2022), related flowsheet synthesis frameworks are described by Barhate et al.

(2025), and Park and Bernal Neira (2026) studied its discrete landscape with Ising-based solvers; we use the corrected-model data of SECQUOIA (2026b). The superstructure contains two reactors in series, each a batch reactor, a CSTR, or a PFR, followed by a batch vaporizer and a crystallizer that is either an unseeded batch cooling crystallizer or a train of one, two, or three mixed-suspension, mixed-product-removal (MSMPR) crystallizers. The external variables ξ_1 and ξ_2 select the two reactors in the order (batch, CSTR, PFR), and ξ_3 selects the crystallizer in the order (batch, one, two, three MSMPR stages), which gives $3 \times 3 \times 4 = 36$ configurations, all logically admissible. These choices are categorical, so this case does not meet the ordering condition of the external-variable reformulation; we index the alternatives in the order of the source enumeration, and the neighborhoods of `D-SDA` and `NOMAD` and the surrogate models act on this order.

For each configuration, `NOMAD` optimizes 12 to 16 normalized operating inputs, including reactor temperatures and residence times, the reagent concentration, the cooling profile, and the crystallizer residence times, with 60 simulations, which solves Problem (Sim) approximately. The objective $-(\dot{m}/10^{-4} + 2d_{50})$, with the production rate $\dot{m}$ in kg/s and the median crystal size d_{50} in μm, rewards both quantities, and a solubility path constraint prevents crystallization before the crystallizer; all 36 configurations satisfy it. These inner optimizations take 91 to 1,019 s per configuration, so evaluating every configuration takes 5.3 h. All methods read the stored results of these inner optimizations, so they evaluate the same $\Phi(\boldsymbol{\xi})$, and each evaluation is charged its recorded inner-optimization time. The reference configuration is a PFR followed by a batch reactor and the batch crystallizer, $\boldsymbol{\xi} = (3, 1, 1)$, with $f_{\text{ref}} = -487.842$; the second-best configuration replaces the PFR by a CSTR and lies 0.8% above it. The budgets are $B = 10$, 15, and 20, with the shared initialization, seeds, unaccepted-response values, and target rule of the process benchmarks, and `D-SDA` uses the same unit L_∞ neighborhood. Supplementary Section S5 lists the 36 objective values and the data provenance.

3.3 Controlled benchmark: Grid-GDP

To study search behavior without NLP subproblems, we also consider 21 two-dimensional test functions adapted from the Quantum Hamiltonian Descent benchmark collection (Wu Group, n.d.). Each is sampled at 31 equally spaced values per coordinate, giving 961 configurations. For each benchmark function, let $[\ell_x, u_x]$ and $[\ell_y, u_y]$ denote the coordinate bounds, and let $I = J = \{0, \dots, 30\}$ be the corresponding index sets. The Grid optimizers receive $(i, j) \in \{0, \dots, 30\}^2$ directly. Relative to the one-based convention in Section 2.1, $(\xi_1, \xi_2) = (i + 1, j + 1)$ is only a relabeling of alternatives. `FMQA-SA` encodes each Grid index in five unsigned bits and rejects decoded values of 31. The physical coordinates are defined as

$$a_i = \ell_x + \frac{i(u_x - \ell_x)}{30}, \qquad b_j = \ell_y + \frac{j(u_y - \ell_y)}{30}. \tag{6}$$

Let f denote the original analytical objective function and let L and U be its minimum and maximum over the 31×31 lattice. Both constants are obtained by exhaustive preprocessing of all 961 lattice points prior to any search. A normalized target tolerance of 10^{-5} corresponds to $10^{-5}(U - L)$ in raw objective units, so equal normalized tolerances do not need to define equally large target sets. Supplementary Table S1 gives target-set sizes for all functions. Full-precision normalization constants are available in the repository (Supplementary Section S7). Each benchmark function is represented by the following GDP:

$$\begin{aligned}
\min_{x,y,Y^x,Y^y} \quad & F(x,y) = \frac{f(x,y) - L}{U - L} \\
\text{s.t.} \quad & \bigvee_{i \in I} \begin{bmatrix} Y_i^x \\ x = a_i \end{bmatrix}, \qquad \bigvee_{j \in J} \begin{bmatrix} Y_j^y \\ y = b_j \end{bmatrix}, \\
& \text{ExactlyOne}(Y_0^x, \dots, Y_{30}^x), \\
& \text{ExactlyOne}(Y_0^y, \dots, Y_{30}^y), \\
& \ell_x \le x \le u_x, \qquad \ell_y \le y \le u_y, \\
& Y_i^x, Y_j^y \in \{\text{False}, \text{True}\}.
\end{aligned} \tag{7}$$

The two exclusive disjunctions contain 62 coordinate-selection Boolean indicators. The objective retains the explicit mathematical expression. The conventional GDP methods therefore receive the expression and its

disjunctive structure, whereas the DFO methods observe only the objective values at the coordinates they select.

Fixing both coordinate disjunctions fixes x and y, leaving no remaining original continuous degrees of freedom. The measured fixed-point `IPOPT` (Wächter and Biegler, 2006) evaluations therefore consist of model preparation, solver invocation, and verification. The unfixed objective functions remain nonlinear and generally nonconvex, so Big-M and Hull produce MINLP representations. The complete mathematical definitions and coordinate domains of the 21 benchmark functions are provided in Supplementary Sections S1-S2 (Table S1) (SECQUOIA, 2026a; Wu Group, n.d.).

3.4 Search protocol and compared algorithms

3.4.1 Shared initialization and evaluation budgets

All configuration-search methods and sampling controls use precomputed configuration responses during the search. For process cases, accepted responses are objective values from `IPOPT` under the initialization and acceptance rules in Section 3.5. Because `IPOPT` returns local solutions, these values can exceed the subproblem value $\Phi(\boldsymbol{\xi})$.

Each Grid-GDP benchmark uses ten seeds, each defining a different initial design, and evaluation budgets $B \in \{10, 30, 50, 70, 100\}$. The primary Grid comparison comprises six search implementations and two sampling controls, giving $21 \times 10 \times 5 \times 8 = 8{,}400$ runs. The process benchmarks comprise 14 model-budget conditions, each evaluated with ten paired seeds, giving 140 runs per method and 980 runs across the six search implementations and uniform sampling (Table 1). The `Lomustine` benchmark adds three budgets with ten seeds each, 210 runs in total. For each problem-budget-seed combination, three distinct initial configurations are generated using seeded Latin hypercube sampling, mapped to integer coordinates, and shared in the same order across all methods and sampling controls. This common design controls the first three evaluations; subsequent sampling, response handling, and stopping rules follow the individual implementations. All process cases use seeds 0-9, with no replacement of failed starts.

Methods may terminate before exhausting B; only Grid `D-SDA` receives the reference target as a stopping criterion. Detailed settings and worker time limits are provided in Supplementary Section S3. Every objective evaluation consumes budget, including initialization, repeated configurations, and unaccepted evaluations; internal cache lookups that do not request an evaluation are free. Target attainment uses the recorded acceptance status and original objective values, while each method receives the following values for unaccepted evaluations. In all process cases, `D-SDA`, `RBFOpt`, `NOMAD`, and `MISO` assign a penalty of 10^6 to unaccepted evaluations. `BAM` represents these evaluations by `NaN`, indicating that no accepted objective value is available. For surrogate training, `FMQA-SA` scales accepted objective values to $[0, 1]$ and assigns unaccepted evaluations a value of 2.

3.4.2 BAM

`BAM` uses local algebraic surrogate models optimized by `BARON` to generate candidate configurations (Ma et al., 2023; The Optimization Firm, 2026). All external variables are integers. The three shared configurations and their responses are supplied without an additional starting point. The remaining evaluation budget is $B - 3$, with other settings at their defaults. On Grid-GDP, the next ten planned evaluations use a Sobol sequence, which distributes points across the domain, subject to the remaining budget. In the process experiments, `BAM` stops at its own evaluation limit after the B-th evaluation.

3.4.3 Budget-limited D-SDA

We use a budget-limited implementation of the previously developed `D-SDA` (Liñán et al., 2020), applying its neighborhood search and discrete line search over the GDP external integer lattice as in LD-SDA (Ovalle et al., 2025). It evaluates the unit L_∞ neighborhood in fixed enumeration order, selects the best improving neighbor, and continues along the resulting direction through a discrete line search. Neighborhood and line-search moves require an absolute objective improvement greater than 10^{-5}. As in the original `D-SDA` and LD-SDA implementations, a configuration is never evaluated twice: when a neighbor or line-search point was

already evaluated in the same run, including the shared initial configurations, `D-SDA` reuses its stored value, and no evaluation is charged. If the evaluation budget B is exhausted, the best solution found is retained.

For the process benchmarks, `D-SDA` starts with the best accepted configuration among the three initial points evaluated and stops when this descent ends. For Grid-GDP benchmarks, initial configurations are ranked by objective value and successive descents are attempted while evaluation budget remains. After each completed descent, the search stops if the best observed objective satisfies the common target criterion in Equation (8); otherwise, it tries the next ranked initial configuration. For all process benchmarks, `D-SDA` searches within the declared integer bounds without precomputed feasibility filtering, using the evaluation accounting and unaccepted-response values stated above. To check that the single process descent does not handicap `D-SDA`, we repeated the 140 process runs with the Grid restart rule, and with further descents from uniformly drawn unvisited configurations until the budget is exhausted (Supplementary Section S12).

3.4.4 FMQA-SA

`FMQA-SA` uses the FM training and QUBO construction code of SECQUOIA (n.d.). At each iteration, a new second-order rank-eight FM is fitted to all evaluated configurations. Finite precomputed responses are min-max scaled to $[0, 1]$ (or zero when all finite values are identical), while non-finite evaluations receive a training target of 2.0.

Integer coordinates are encoded in unsigned binary, and the fitted FM is converted to a QUBO. Binary candidates are generated on CPU using D-Wave's classical simulated annealing sampler (D-Wave Systems, n.d.). The lowest-energy decoded candidate within the coordinate bounds that has not been previously evaluated is selected; if none is available, a candidate is drawn uniformly from the remaining unevaluated configurations. Candidates are not screened for the feasibility of logical or nonlinear constraints before evaluation. The new observation is appended, and FM fitting and candidate generation repeat until the evaluation budget B or the 1,800 s method time limit is reached.

3.4.5 MISO

`MISO` uses the CPTV strategy of Müller (2016) as implemented in `soogo` 2.1.0, with a cubic radial basis function model with a linear polynomial tail, integer coordinates, and a median low-pass filter on large objective values. The three shared configurations and their responses initialize the model. Because `soogo` 2.1.0 does not accept pure-integer domains in its coordinate-perturbation step, our adapter applies the same dynamically dimensioned search perturbation on the integer lattice and disables the continuous local search, which has no continuous coordinates to act on. A linear tail needs $d + 1$ affinely independent points, so in the three-dimensional `Small Batch` case uniformly drawn admissible configurations, charged to the budget, are added until the design spans the space; all other cases are two-dimensional. A run stops at the budget, or after ten consecutive iterations without a new evaluation when no unevaluated candidate remains.

3.4.6 NOMAD

`NOMAD` 4.6.0 (Audet et al., 2022), called through `PyNomadBBO`, runs mesh adaptive direct search with all external variables declared integer. It starts from the best of the three shared initial configurations, whose stored value it reuses without charge, with other settings at their defaults. With integer variables, `NOMAD` stops when its mesh reaches the integer resolution, so, like `D-SDA`, it can end before the budget. `NOMAD` 4.6.0 sometimes terminated with a segmentation fault after its last new evaluation. Such a run is repeated with larger evaluation caps, and a rerun is accepted only if it evaluates exactly the same configurations in the same order; 123 of the 1,190 runs completed this way. Twenty-one Grid runs at $B = 100$ crashed on every attempt after identical evaluations; they are scored on the evaluations completed before the crash, and 16 of them had already reached the target (Supplementary Section S11).

3.4.7 RBFOpt

`RBFOpt` uses MSRSM, automatic RBF selection, and the default genetic candidate-search option of version 4.3.0 (Costa and Nannicini, 2018; Nannicini, 2021). All external variables are integers, and the three shared initial configurations are supplied with the additional package initialization disabled. Search stops at the

evaluation budget or the native iteration and stall limits. Other surrogate-minimization operations use standalone `Bonmin` 1.8.9 (Bonami et al., 2008) and `IPOPT` 3.14.19 (Wächter and Biegler, 2006) executables. These calls contribute to search time, but not the objective-evaluation budget.

3.4.8 Sampling controls and full enumeration

Uniform sampling evaluates the same three initial configurations and then draws uniformly without replacement from the remaining declared integer domain. It uses a separate deterministic random stream, no objective-dependent selection, and no logical or NLP-feasibility screening before evaluation. The Grid `BAM`-prefix control repeats `BAM`'s ten predetermined sampling points in the same order after the three shared starts, then samples uniformly without replacement, stopping at the budget. This control isolates the contribution of the fixed initial design of `BAM`. Full enumeration evaluates every configuration of the DFO domain once. It reaches the target by construction, because f_{ref} is the best accepted value among these configurations, and its cost is the sum of their measured point costs.

3.5 Execution environment and conventional GDP comparisons

All experiments were conducted on a server with an AMD EPYC 7643 processor (48 cores, 2.3 GHz) and 1 TB RAM. All optimization methods and numerical libraries were configured for single-threaded execution. The software environment included `Pyomo` 6.10.0 (Bynum et al., 2021), `RBFOpt` 4.3.0, `BAM` 5.0.0, `PyTorch` 2.8.0 (Paszke et al., 2019), and `SCIP` 9.2.4 (Bolusani et al., 2024). We used the General Algebraic Modeling System (`GAMS`) 52.2.0 (GAMS Development Corporation, 2025) to call `IPOPT` and `SCIP`. ChatGPT and Codex (OpenAI), including GPT-6 Astra, and Claude Code (Anthropic), with Claude Opus 5.5, assisted in developing analysis, plotting, and experiment code. The authors reviewed the code and checked the generated tables and figures against the recorded experimental results.

Fixed-disjunction NLP evaluations. We fix the original Boolean GDP decisions and evaluate the resulting subproblems using `IPOPT` 3.14.19 with `MUMPS` 5.8.1 (Amestoy et al., 2001). For the Grid-GDP benchmarks, all 20,181 configurations were evaluated individually, while the process benchmarks used fixed-configuration evaluations of the corresponding process models. `IPOPT` used a tolerance of 10^{-8} and a solver time limit of 60 s per configuration. Process acceptance requires a finite objective, a maximum absolute violation of any constraint or variable bound not greater than 10^{-5}, and the accepted solver and termination statuses specified in the Supplementary Section S3.1. Time-limited candidates remain unaccepted even when feasible. `CSTR` sizes 5-30 use the mass-balance/zero-recycle initialization. `Column` and `Small Batch` use their saved initialization files, and `Dynamic` uses fresh clones of the unsolved discretized template.

Conventional GDP comparisons. Conventional comparisons include `GDPopt` RIC, GLOA, and LOA, as well as Big-M and Hull reformulations. `GDPopt` uses `SCIP` 9.2.4 for MILP master problems and `IPOPT` 3.14.19 for NLP subproblems (Chen et al., 2022; Pyomo Development Team, n.d.). Because fixing both Grid disjunctions also fixes x and y, the Grid subproblems are solved as linear problems by `SCIP`, and `IPOPT` is not called in those runs. The Big-M and Hull reformulations are solved directly using `SCIP`. All conventional GDP methods use the original GDP domains, a time limit of 1,800 s per instance, and a relative optimality gap of 10^{-5}.

3.6 Target attainment and cumulative success profiles

Let f_{best} denote the lowest feasible objective value observed so far within the evaluation budget, and let f_{ref} denote the reference objective value. For process benchmarks, f_{best} is computed from evaluations that meet the solver acceptance criteria in Section 3.5. A run reaches the reference target when

$$|f_{\text{best}} - f_{\text{ref}}| \leq 10^{-5} \max\{1, |f_{\text{ref}}|\}. \tag{8}$$

For the process experiments, f_{ref} is the lowest accepted objective value among the precomputed fixed-configuration NLP evaluations, so reaching the target means matching the best configuration that full enumeration of the stored responses would find. Runs that terminate with an execution failure are counted as

unsuccessful, even if the target was observed earlier. For process runs, the reported solution is the best accepted configuration recorded within budget, including the final evaluation. Reported objectives and bounds use six significant figures; all target and bound checks use unrounded values. For the full-process-GDP comparison, a feasible objective at or below $f_{\text{ref}} + 10^{-5}\max\{1, |f_{\text{ref}}|\}$ counts as achievement, so solutions better than the reference are also included.

For each evaluation budget B, let $\mathcal{R}_B$ denote the set of problem-seed instances compared across methods, and let s identify a solution method. For each instance $r \in \mathcal{R}_B$, H_{sBr} equals one if the run reaches the target and zero otherwise. Let E_{sBr} be the index of the first objective evaluation at which the target is reached, with $E_{sBr} = +\infty$ if the target is not reached. At a threshold of N objective evaluations, where $0 \le N \le B$, the absolute evaluation-count profile is

$$C_{sB}^{\text{eval}}(N) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ E_{sBr} \le N\}. \tag{9}$$

Here, $\mathbf{1}\{\cdot\}$ equals one when the stated conditions hold and zero otherwise, and $|\mathcal{R}_B|$ is the number of problem-seed instances. Thus, $C_{sB}^{\text{eval}}(N)$ gives the fraction of runs that reach the target within N evaluations. For the Grid-GDP benchmarks, $|\mathcal{R}_B| = 210$ at each budget; for each process benchmark and budget, $|\mathcal{R}_B| = 10$. All runs, including target misses, remain in the denominator.

Experiments at different evaluation budgets are analyzed separately, so a larger-budget trajectory need not extend a smaller-budget one. These profiles count raw evaluations, without the best-solver normalization of performance profiles (Dolan and Moré, 2002) or the dimension normalization of data profiles (Moré and Wild, 2009).

NLP subproblems solved. An evaluation counts as an NLP subproblem solve when its configuration passes logical and FBBT preprocessing and `IPOPT` is called, whether or not the result is accepted. For `GDPopt`, we count the `IPOPT` calls confirmed in its execution logs. Internal NLP counts are unavailable for the direct Big-M and Hull solves.

3.7 Runtime accounting and uncertainty

For a recorded lookup-based run with n objective evaluations and measured lookup-call times, the estimated full-run completion time is

$$\widehat{T}_{\text{complete}} = T_{\text{recorded}} - \sum_{q=1}^{n} t_q^{\text{lookup}} + \sum_{q=1}^{n} c(\boldsymbol{\xi}^{(q)}) + t_{\text{prep}}. \tag{10}$$

Here, T_{recorded} is the measured runtime of the search based on lookup, t_q^{lookup} is the measured lookup-processing interval of its qth objective call, and $\boldsymbol{\xi}^{(q)}$ is the external integer configuration evaluated in that call. The point cost $c(\boldsymbol{\xi}^{(q)})$ is measured separately for the same benchmark model, problem size and configuration, including configuration-specific preparation, preprocessing, invocation of the solver when required and validation. The term t_{prep} accounts for the preparation of a single model that is not already included in the point costs. Search time includes surrogate fitting, candidate generation, auxiliary optimization, and recording performed within the method clock. Throughout this manuscript, full-run completion denotes completion of this recorded search stage with reconstructed evaluation costs, excluding final result extraction and export. Every recorded objective call contributes its point cost, including initialization, repeated visits, and preprocessing rejections; the same stored measurement is used each time a configuration is revisited. Completion times therefore combine recorded search paths with one stored cost measurement per configuration.

Grid-GDP timers. Grid-GDP enumeration and objective normalization are excluded from the timing analysis, and $t_{\text{prep}} = 0$ because configuration-specific preparation is included in point costs. Recorded lookup-call times are subtracted from the method runtime for all compared Grid methods. The 21 `NOMAD` runs that crashed on every attempt have no recorded times and are left out of the Grid timing summaries.

Process timers. Structurally rejected configurations incur measured preparation and rejection costs without an NLP solution (Supplementary Section S3.1). For `Dynamic`, the original model-template construction and discretization cost is included once per run through t_{prep}; additional campaign-level preparation is not charged repeatedly. All in-budget evaluations have measured point costs, and estimated full-run completion times are available for all 980 primary process runs.

Conventional GDP completion times are measured separately and are not augmented with the DFO point costs. For Grid-GDP, conventional GDP timings include per-run preparation, reformulation, solution, and validation, but exclude shared-template preparation and lattice enumeration. Full-process-GDP timings include model-template preparation.

Statistical comparisons. With ten seeds per process condition, we compare two methods with the two-sided exact McNemar test on their seed-paired target outcomes, and we call differences that this test does not resolve at the 0.05 level comparable (Supplementary Table S10). The tests are descriptive and not adjusted for multiple comparisons. Grid comparisons use a percentile bootstrap that resamples the 21 functions (Supplementary Section S9).

4 Computational Results and Discussion

4.1 Process GDP case studies

4.1.1 Target attainment

Table 2 reports target attainment for the 14 process conditions. Across the 60 paired `CSTR` runs, `RBFOpt` reaches the target in 45, `MISO` in 43, `NOMAD` in 21, and `D-SDA` in 16. `RBFOpt` succeeds in 30 runs in which `D-SDA` misses, and `D-SDA` in one in which `RBFOpt` misses (exact paired test $p < 10^{-7}$; Supplementary Table S10). With ten seeds per size, the difference is significant at the 0.05 level only for `CSTR5` and `CSTR30`, but it has the same sign at every size. On `Dynamic` at $B = 12$, `RBFOpt` reaches all ten targets and `D-SDA` four ($p = 0.031$); `D-SDA` reaches all ten from $B = 20$ onward.

`MISO` shows the same advantage over `D-SDA` with a different profile across sizes. It succeeds in 28 `CSTR` runs in which `D-SDA` misses, against one in the other direction ($p < 10^{-6}$), and it reaches all 30 targets on `CSTR20` through `CSTR30`, but only 13 of 30 on `CSTR5` through `CSTR15`, where `RBFOpt` reaches 21. Pooled over the `CSTR` sizes, `MISO` and `RBFOpt` disagree in 16 runs, seven in favor of `MISO` ($p = 0.80$). `NOMAD` reaches more `CSTR` targets than `D-SDA` (7 against 2 discordant runs, $p = 0.18$) but fewer than `RBFOpt` (5 against 29, $p < 10^{-4}$). Like `D-SDA`, it stops before the budget in many runs (77 of 140) because its mesh reaches the integer resolution. Exploratory per-condition tests of `NOMAD` and `MISO` against `D-SDA`, `RBFOpt`, and each other find no contrast that remains significant after a Holm correction over the 70 tests (Supplementary Section S11).

On the two smallest design models, `D-SDA` and `RBFOpt` are comparable. `D-SDA` reaches five `Column` targets and `RBFOpt` four ($p = 1$), and on `Small Batch` they reach eight and five ($p = 0.38$). `MISO` reaches all ten `Column` targets and `NOMAD` seven; on `Small Batch` they reach seven and six. `BAM` reaches all ten `CSTR5` targets, compared with eight for `RBFOpt` ($p = 0.50$). The only `Column` hits of `BAM` and uniform sampling occur in seed 6, whose shared initial design contains the target.

The single-descent `D-SDA` stops before exhausting its budget in 87 of the 140 process runs. Restarting it from the remaining shared initial configurations, or from uniformly drawn unvisited configurations until the budget is spent, adds one target, raising target attainment from 79 to 80 of 140 runs; both restart variants improve the final best objective in 14 runs (Supplementary Section S12).

`FMQA-SA` reaches 42 of 140 process targets, compared with 37 for uniform sampling. In the small process domains, the 100 annealed samples often contain no new configuration within the bounds, so 52.9% of the `FMQA-SA` proposals come from its uniform fallback (Supplementary Section S3.2).

Table 2 also reports search times, which exclude configuration-evaluation costs. Median search times stay below 0.06 s for `D-SDA` and uniform sampling and below 1 s for `BAM`. For `RBFOpt`, they grow with the budget, from 0.06 s on `CSTR5` to 26.3 s on `Dynamic` at $B = 50$. `MISO` needs 0.66 to 158 s because it refits its model and scores a large candidate pool at every iteration, whereas `NOMAD` stays below 2 s. `FMQA-SA` needs 3.7 to 29.8 s because it refits its factorization machine after every evaluation.

Table 2: Reference-target attainment of the seven DFO methods in the `IPOPT`-table process experiments. Each cell lists target hits out of ten paired seeds, median search time, and [Q25, Q75] search time in seconds, from top to bottom. Search times exclude configuration-evaluation costs and summarize all ten seeds. The quartile ranges describe variability across seeds. Bold counts mark the largest number of target hits in each row, including ties. D: include median and interval search times (not accounting for NLP solutions) for each method A: Added median and [Q25, Q75] search times from the completed 700-run comparison, excluding NLP evaluation costs and retaining all ten seeds per condition. C: Added `NOMAD` and `MISO` from the audited Dantzig return (980 runs in total).

Case	B	`D-SDA`	`NOMAD`	`RBFOpt`	`MISO`	`BAM`	`FMQA-SA`	Uniform
		5	7	4	**10**	1	4	1
`Column`	13	0.0120	0.0423	0.226	0.672	0.0177	6.85	0.00716
		[0.0116, 0.0124]	[0.0358, 0.0461]	[0.205, 0.239]	[0.669, 0.681]	[0.0168, 0.155]	[6.81, 6.89]	[0.00711, 0.00748]
		2	4	8	4	**10**	3	4
`CSTR5`	8	0.0105	0.0367	0.0596	0.659	0.0360	3.66	0.00460
		[0.0102, 0.0111]	[0.0323, 1.13]	[0.0590, 0.0597]	[0.658, 0.669]	[0.0151, 0.0816]	[3.64, 3.67]	[0.00387, 0.00478]
		3	3	**6**	2	1	2	1
`CSTR10`	15	0.0137	1.83	0.302	0.677	0.137	8.00	0.00807
		[0.0130, 0.0147]	[0.469, 1.88]	[0.288, 0.315]	[0.675, 0.683]	[0.126, 0.158]	[7.98, 8.08]	[0.00772, 0.00812]
		2	2	**7**	**7**	1	1	2
`CSTR15`	22	0.0169	0.825	1.00	3.29	0.150	12.4	0.0113
		[0.0156, 0.0172]	[0.0949, 1.82]	[0.492, 3.26]	[1.21, 7.50]	[0.140, 0.162]	[12.3, 12.4]	[0.0108, 0.0116]
		3	2	7	**10**	2	1	2
`CSTR20`	29	0.0207	1.63	2.82	10.8	0.167	16.9	0.0139
		[0.0189, 0.0218]	[0.521, 1.92]	[0.632, 4.01]	[9.02, 15.8]	[0.165, 0.313]	[16.8, 16.9]	[0.0121, 0.0148]
		3	6	8	**10**	2	1	1
`CSTR25`	36	0.0258	1.08	2.47	17.6	0.401	21.3	0.0161
		[0.0238, 0.0299]	[0.170, 2.32]	[1.86, 4.43]	[16.4, 18.8]	[0.392, 0.449]	[21.3, 21.4]	[0.0147, 0.0186]
		3	4	9	**10**	2	1	1
`CSTR30`	43	0.0585	1.90	4.99	18.1	0.425	25.8	0.0241
		[0.0556, 0.0610]	[1.76, 7.65]	[2.95, 10.3]	[14.0, 19.3]	[0.404, 0.451]	[25.7, 26.0]	[0.0224, 0.0257]
		8	6	5	7	2	6	4
`Small Batch`	10	0.0102	0.0475	0.170	0.668	0.0169	4.87	0.00536
		[0.00971, 0.0103]	[0.0333, 0.0540]	[0.159, 0.180]	[0.666, 0.670]	[0.0161, 0.0191]	[4.86, 4.89]	[0.00506, 0.00545]
		4	8	**10**	**10**	0	2	0
`Dynamic`	12	0.0119	0.0419	0.186	0.671	0.0173	6.16	0.00589
		[0.0113, 0.0125]	[0.0377, 0.0630]	[0.182, 0.189]	[0.670, 0.676]	[0.0172, 0.0174]	[6.14, 6.20]	[0.00540, 0.00658]
		6	**10**	**10**	**10**	3	3	3
`Dynamic`	15	0.0123	0.0896	0.296	0.679	0.139	8.04	0.00740
		[0.0115, 0.0131]	[0.0627, 1.09]	[0.291, 0.321]	[0.676, 0.683]	[0.128, 0.156]	[8.01, 8.09]	[0.00622, 0.00760]
		10	**10**	**10**	**10**	3	3	3
`Dynamic`	20	0.0133	1.49	0.367	25.0	0.143	11.2	0.00987
		[0.0119, 0.0154]	[0.447, 6.90]	[0.355, 0.381]	[24.6, 30.1]	[0.130, 0.156]	[11.1, 11.2]	[0.00927, 0.00991]
		10	**10**	**10**	**10**	4	4	4
`Dynamic`	30	0.0149	1.77	7.61	79.7	0.355	17.4	0.0141
		[0.0124, 0.0157]	[1.44, 9.24]	[4.88, 13.7]	[79.4, 80.5]	[0.204, 0.364]	[17.2, 17.5]	[0.0135, 0.0145]
		10	**10**	**10**	**10**	7	5	5
`Dynamic`	37	0.0134	1.77	10.1	81.0	0.376	21.7	0.0173
		[0.0118, 0.0160]	[1.43, 9.23]	[5.25, 14.2]	[79.7, 82.9]	[0.364, 0.383]	[21.6, 21.8]	[0.0171, 0.0174]
		10	**10**	**10**	**10**	**10**	6	6
`Dynamic`	50	0.0135	1.77	26.3	158	0.732	29.8	0.0233
		[0.0129, 0.0153]	[1.42, 9.21]	[17.0, 37.2]	[157, 160]	[0.685, 0.751]	[29.7, 30.1]	[0.0229, 0.0235]

Supplementary Figure S1 shows the evaluation at which each successful run first reaches the target.

4.1.2 NLP subproblems solved

Figure 1 compares the number of NLP subproblems that each run solves with the number that full enumeration of the configuration domain requires. On `CSTR5` through `CSTR20` and on `Dynamic`, RIC and LOA solve exactly as many subproblems as full enumeration, and their execution logs show that every subproblem fixes a different configuration (Supplementary Section S10). RIC also enumerates all 325 admissible `CSTR25` configurations and solves 455 of the 465 `CSTR30` subproblems before the time limit. The DFO methods solve at most 38 subproblems in any run, and `MISO` and `RBFOpt` reach the `CSTR30` target in ten and nine of ten runs with medians of 29.5 and 31.5, 6 to 7% of the enumeration count.

Enumeration does not guarantee that a logic-based method reaches the target. RIC and LOA miss the `CSTR10` target after solving all 55 of its subproblems, and RIC misses the `CSTR25` target after solving all 325. These misses come from local NLP solutions obtained with the `GDPopt` initialization, which differs from the initialization used to build the lookup tables (Supplementary Section S3.1). GLOA solves two to four subproblems per `CSTR` model and misses every `CSTR` target.

D: If the NLPs or times are comparable to the times in Figures 2 and 3, include them as a vertical bar to contextualize the result (vertical bar is equivalent to them having solved the problem). Units for time (s) are missing. Highlight the best result per row in bold. If the time limit is reached, then put the time as 1800+ (in some cases it is $1791 \leq 1800$) A: Added the joint completion-time comparison to Figure 3 and moved the detailed table to the supplement, retaining all DFO seeds and single conventional observations. Time units and bold row minima are shown; TL identifies limit terminations while preserving measured times rather than replacing them with 1800. Removed duplicate supplementary panels and the unused cumulative time-profile definition. C: Figure 1 now places the `GDPopt` NLP counts on the same axis as the DFO runs, with full enumeration as the vertical reference line; Table S15 shows time-limited runs as 1800+.

4.1.3 Completion time

In wall time, the advantage of fewer subproblem solves depends on how much each solve costs. Configuration evaluations have median costs of 0.4 to 2.7 s in these models (Supplementary Table S12). Full enumeration therefore takes 10.2 s for `Small Batch`, 47.2 s for `Dynamic`, 137 s for `Column`, and 1,719 s for `CSTR30` (Supplementary Table S5). On `Small Batch`, the DFO medians of 3.49 to 9.09 s are close to the enumeration time, and on `Dynamic` at $B = 50$, `RBFOpt` and `MISO` need 65.14 and 195.36 s, more than enumerating all 72 configurations. On `CSTR30`, `MISO` and `RBFOpt` reach the target in ten and nine of ten runs with medians of 107.33 and 101.96 s, about one sixteenth of the enumeration time, while `NOMAD` and `D-SDA` reach four and three targets with medians of 71.23 and 47.17 s.

Search overhead and stopping rules also shape completion times. For `Dynamic` at $B = 50$, search overhead contributes 42.7% of the mean completion time of `RBFOpt`, compared with less than 0.1% for `D-SDA`, and `MISO` spends a median of 158 s of its 195 s in search. Across these ten runs, `D-SDA` and `NOMAD` evaluate 167 and 170 configurations because they stop at a local optimum or at the integer mesh resolution, whereas `RBFOpt`, `MISO`, `BAM`, and `FMQA-SA` each evaluate 500. At $B = 30$, `RBFOpt` reaches all ten `Dynamic` targets in fewer evaluations than `D-SDA` (Supplementary Figure S1), but it continues to the budget, and its median completion time is 34.19 s, compared with 20.02 s for `D-SDA`, 22.00 s for `NOMAD`, and 106.24 s for `MISO`. On `Column`, where `MISO` reaches all ten targets, the median completion times are 31.93 s for `NOMAD`, 31.98 s for `MISO`, 35.44 s for `D-SDA`, and 54.60 s for `RBFOpt`.

Figure 2 places the conventional GDP methods on the same time axis, and Supplementary Table S15 lists their objectives; Supplementary Figure S2 shows the other eight conditions. Hull reaches the target on seven of the nine models, RIC, LOA, and Big-M on five each, and GLOA only on `Small Batch`. Big-M and Hull solve `Small Batch` in about 0.5 s, faster than any DFO run, and they are the only conventional methods that reach the `CSTR30` target, both stopping at the 1,800 s time limit. No conventional method reaches the `Column` target. D: you usually do not use fractions in a sentence, and the sentence just reads like a laundry list. Have more prose A: Replaced the fraction list with a prose summary of the updated results.

All six `CSTR` GLOA runs report `optimal` termination, yet none comes within the relative target tolerance of 10^{-5} (Equation (8)); their objectives exceed the references by 2.2% on `CSTR5` and by 32% on `CSTR30`. D: help the reader, remind them what this means and the epsilon it represents A: Stated the reference-target

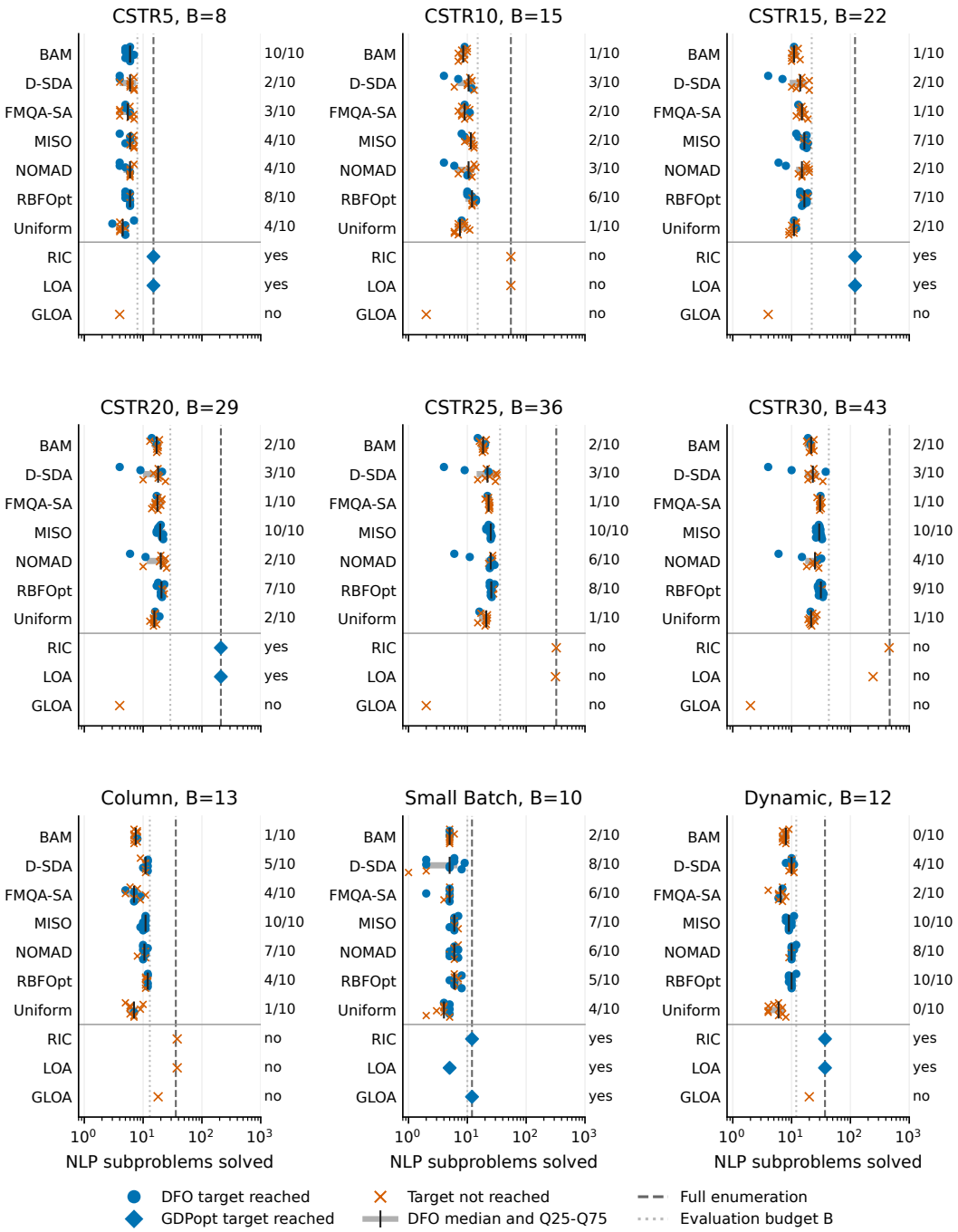


Figure 1: NLP subproblems solved per run for the nine process models, with `Dynamic` at $B = 12$. Dots show successful DFO seeds, diamonds successful `GDPopt` runs, and crosses target misses; gray segments and ticks show Q25–Q75 and medians over the ten DFO seeds. The dashed line marks the subproblems needed to evaluate every configuration of the DFO domain, and the dotted line the evaluation budget B. Logical and FBBT rejections are not counted as NLP solves. `GDPopt` searches the full `Column` box of 225 configurations, so its count can exceed the enumeration line of the 64-configuration DFO domain. Direct Big-M and Hull runs are omitted because their internal NLP counts are unavailable.

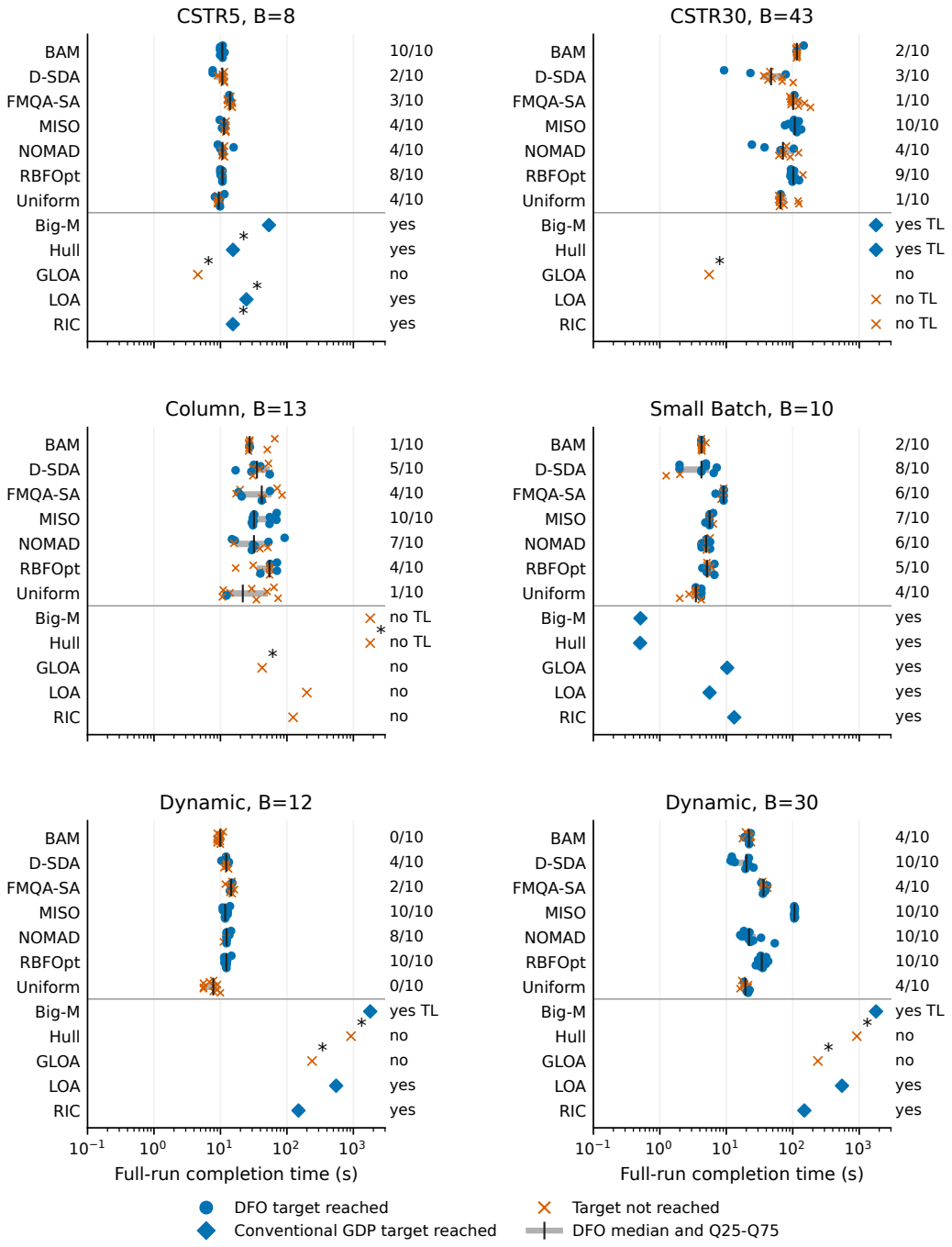


Figure 2: Joint full-run completion-time comparison for six process conditions. DFO times are reconstructed using Equation (10); conventional GDP times are measured and include model preparation. Dots show individual successful DFO runs, diamonds successful single conventional runs, and crosses target misses under the rules in Section 3.6. Gray segments and ticks show Q25–Q75 and medians across all ten DFO seeds; right-hand labels give hits out of ten or the single-run outcome. TL denotes reported time-limit termination; asterisks mark the bound inconsistencies discussed in Section S8 of the supplement. The two Dynamic panels reuse the same conventional observations.

tolerance explicitly. Their reported lower bounds exceed independently checked feasible objectives. An earlier configuration that used `SCIP` as a global solver for both master problems and subproblems returned the same six objectives within a relative difference of 3×10^{-5}, so these misses do not come from the local NLP solver. Supplementary Section S8 reports these and the other bound inconsistencies, which affect the interpretation of reported optimality but not the target counts. D: Why would a reader know that Dantzig is a computer? SInce we refer to all computational results from that computer, you can just avoid mentioning its name A: Removed the machine name.

4.2 Simulation-based lomustine case

Table 3 reports the `Lomustine` results. `RBFOpt` and `MISO` reach the reference configuration in eight and seven of ten runs with ten evaluations and in all ten runs with 20, and their successful runs find it at a median of the fifth to sixth and the sixth to seventh evaluation. `NOMAD` reaches it in six runs at every budget because, when the budget allows more, it stops after a median of 12.5 evaluations. `D-SDA` reaches it once with ten evaluations: its unit L_∞ neighborhood in three dimensions contains up to 26 configurations, so ten evaluations do not complete the first neighborhood search. `BAM` misses every target with ten evaluations, and all ten of its runs end at the same PFR–PFR configuration, 11% above the reference. Of the 96 misses over all methods and budgets, 41 end at this PFR–PFR configuration and 27 at the CSTR–batch configuration, 0.8% above the reference.

In this case, an evaluation costs minutes of simulation, so the completion time is governed by the number of evaluations. Median search times stay below 13 s, whereas the median completion times with ten evaluations range from 1.0 to 1.5 h, compared with 5.3 h for evaluating all 36 configurations. `MISO`, which spends the most search time per evaluation on the other process models, completes its ten-evaluation runs in a median of 1.13 h, about as fast as any method.

Table 3: `Lomustine`: runs out of ten that reach the reference configuration (top) and median full-run completion time in hours (bottom), including target misses. Bold counts mark the largest number of hits for each budget, including ties. Each evaluation is charged the recorded wall time of the PharmaPy simulation-based optimization of its configuration.

Method	$B = 10$	$B = 15$	$B = 20$
`D-SDA`	1	4	4
	1.15	1.68	1.97
`NOMAD`	6	6	6
	1.05	1.22	1.22
`RBFOpt`	**8**	9	**10**
	1.24	1.70	2.44
`MISO`	7	**10**	**10**
	1.13	1.82	2.66
`BAM`	0	4	6
	1.36	1.90	2.48
`FMQA-SA`	2	3	7
	1.47	2.39	3.09
Uniform	2	4	5
	1.48	2.11	2.91

4.3 Controlled search behavior on Grid-GDP

Grid-GDP isolates search behavior on explicit two-dimensional functions whose configurations need no NLP solve. Table 4 reports the separate experiments at the five budgets. `MISO` reaches the largest target fraction at $B = 30$, 70, and 100, while `RBFOpt` leads at $B = 50$ and `BAM` at $B = 10$. At $B = 100$, `MISO` and `RBFOpt` reach 86.2% and 81.4% of the targets, `NOMAD` 60.0%, and `D-SDA` 48.1%, with `BAM` and `FMQA-SA` in between and uniform sampling at 21.4%. The `NOMAD` fraction at $B = 100$ includes 16 targets reached before a reproducible

solver crash; counting those 21 crashed runs as misses would give 52.4%. The paired bootstrap intervals in Supplementary Section S9 exclude zero for `RBFOpt` against `D-SDA` at both $B = 10$ and $B = 100$.

At $B = 10$, `BAM` and its prefix control each reach 52.9% because nine of the 21 functions have their target at the domain center, which `BAM` evaluates as the fifth point of its Sobol design. By $B = 100$, the model-based search of `BAM` adds 23 hits over the prefix control (Figure 3). `D-SDA` levels off at 48.1% from $B = 70$ onward, once the descents from its three initial configurations end, and its median number of evaluations stays at 37 for every budget from $B = 50$ onward. Excluding Xin-She Yang 3, whose target set covers 870 of the 961 lattice points, and Bohachevsky 2, which nearly duplicates Sum of Squares on the lattice, preserves this ordering.

Table 4: Grid-GDP reference-target hit percentages from the recorded trajectories. Each cell contains 210 runs (21 functions and ten seeds). The evaluation budget includes the three shared initial evaluations and repeated configurations. `BAM`'s 111 hits at $B = 10$ all precede surrogate-guided selection (11 shared-initialization hits and 100 at two predetermined Sobol points). `NOMAD` at $B = 100$ includes 16 hits reached before a reproducible solver crash. Bold entries mark the largest percentage in each row, including ties. D: last sentence does not belong to the caption and highlight the best in each row A: Moved the repository pointer to the text and bolded each row maximum, including ties.

Budget B	`D-SDA`	`NOMAD`	`RBFOpt`	`MISO`	`BAM`	`FMQA-SA`	Uniform	`BAM`-prefix control
10	6.2	18.6	20.0	18.1	**52.9**	7.6	6.7	**52.9**
30	37.1	52.9	61.9	**67.1**	54.8	22.4	11.9	54.8
50	43.3	59.0	**73.3**	70.0	57.6	36.2	13.8	56.7
70	48.1	60.0	76.7	**81.4**	62.9	48.6	17.1	57.1
100	48.1	60.0	81.4	**86.2**	70.0	61.4	21.4	59.0

Because the explicit expression is available, the direct Big-M and Hull reformulations solved with `SCIP` reach all 21 Grid targets in a median of 0.66 and 0.53 s per function, less than the median completion time of every DFO method (Supplementary Table S6). The `GDPopt` methods reach 14 or 15 of the 21 targets, and in diagnostic runs their preprocessing rejected the feasible reference configurations of the three functions that all of them miss. Grid-GDP therefore tests search behavior; for these explicit functions, a direct reformulation is the method of choice.

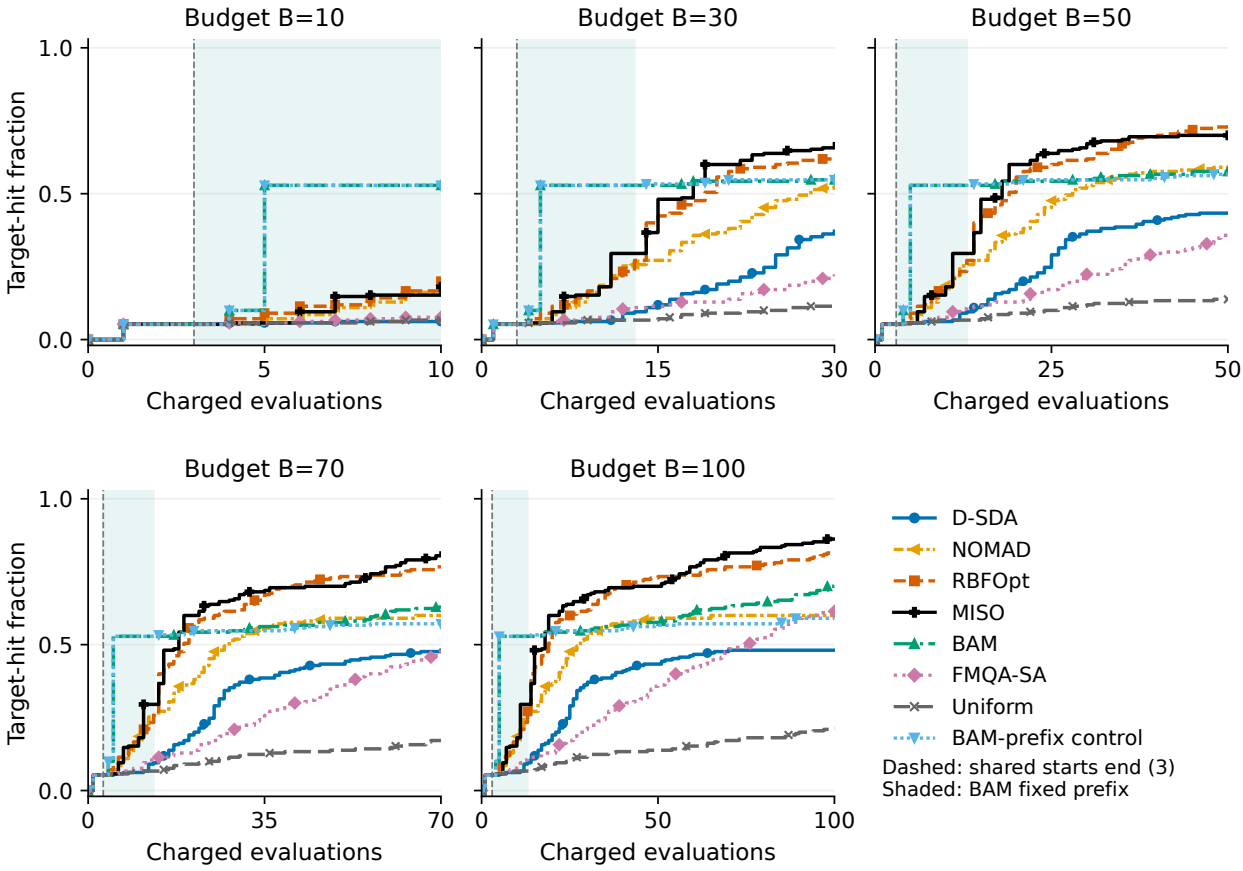


Figure 3: Cumulative success profiles for 21 Grid-GDP functions and ten seeds per function at $B = 100$, following Equation (9). The target is the normalized reference value zero within 10^{-5}, as defined by Equation (8). Each curve includes all 210 runs; initial and repeated evaluations count, and target misses remain in the denominator. The dashed line marks the three shared initial evaluations. Uniform sampling and the `BAM`-prefix control are defined in Section 3.4.8. `BAM` rises early, while `MISO` reaches the largest final fraction at 100 evaluations, followed by `RBFOpt`. D: You cannot differetiate BAM and BAM prefix. I believe this color scheme is not colorblind friendly A: Distinguished BAM and its prefix control by solid versus dash-dot lines, filled versus open markers, and green versus black; curve values are unchanged.

4.4 Discussion

No single method reaches the most targets in every case. The ranking depends on the size of the configuration space, on where the target lies relative to the initial configurations and to the configurations without accepted objectives, and on how each method uses its evaluation budget.

On `CSTR`, the recycle/feed ordering makes roughly half of each configuration box logically inadmissible, and the target lies at the corner (N, N), on the edge of the admissible region. On `CSTR10` through `CSTR30`, 31 of the 36 `D-SDA` misses end at the configuration $(5, 1)$, whose volume of 3.13020 is the same for every N. Because the reference volume decreases with N, this local minimum lies 8.3% above the reference for `CSTR10` and 13.0% above it for `CSTR30`, which explains why the `D-SDA` gaps grow with size. Restarted descents mostly end at $(5, 1)$ or at the target, so the misses reflect the large basin of this local minimum, and additional budget does not remove them. `NOMAD`, which polls configurations around its incumbent, ends 32 of its 33 misses on these sizes at $(5, 1)$ as well. `RBFOpt` and `MISO` can propose configurations far from their incumbents and reach the corner in nine and ten of the ten `CSTR30` runs. A global surrogate does not guarantee success on small budgets, however: `MISO` reaches 13 of the 30 targets on `CSTR5` through `CSTR15`, with 8 to 22 evaluations, compared with 21 for `RBFOpt`. On `Column`, four of the five successful `D-SDA` runs reach the target in one or two improving moves, and the fifth starts at it. With only 29 and 12 accepted configurations and budgets of 13 and 10 evaluations, `Column` and `Small Batch` give a surrogate model few observations to learn from, and local descent remains competitive with `RBFOpt`. `MISO` nevertheless reaches all ten `Column` targets; in the nine runs whose initial design does not contain the target, it finds it between the 7th and 11th of its 13 evaluations. On Grid-GDP, the early lead of `BAM` reflects the symmetry of the test functions with respect to its initial design, and `FMQA-SA` improves on uniform sampling there but not in the small process domains, where its uniform fallback dominates. From $B = 30$ onward, the two radial-basis-function methods, `MISO` and `RBFOpt`, reach the most Grid targets. D: Provide more details here: why do some instances benefit some solvers? What's the hidden structure? A: Added evidence from the saved records on Grid target locations, CSTR30 local stopping, and Column improvement paths.

The methods also use additional evaluations differently. `D-SDA` terminates when its descents reach configurations without an improving neighbor, and `NOMAD` when its mesh reaches the integer resolution, so additional budget helps them only until then. On Grid, `NOMAD` stops after a median of 31.5 evaluations from $B = 50$ onward, and its target fraction stays at 60.0% from $B = 70$ to $B = 100$. The surrogate-based methods and the sampling controls have no comparable termination test and use the full budget in every run, so their Grid target-hit rates keep increasing between $B = 70$ and $B = 100$, by 4.8 percentage points for `RBFOpt` and `MISO`, 7.1 for `BAM`, and 12.9 for `FMQA-SA`, whereas the `D-SDA` rate stays at 48.1%. Once a method has found the target, however, the remaining evaluations add cost without improving the result: `RBFOpt` and `MISO` reach all ten `Dynamic` targets at $B = 12$, yet their median completion times grow from 12.29 and 11.80 s at $B = 12$ to 65.14 and 195.36 s at $B = 50$, whereas `NOMAD`, which stops early, needs 21.99 s at $B = 50$. Additional search time also does not guarantee more targets: on Grid at $B = 100$, `FMQA-SA` spends a median of 62.04 s on search and reaches fewer targets than `BAM`, which spends 0.43 s. For `MISO`, it does: with a median of 32.70 s, about four times the search time of `RBFOpt`, it reaches 181 targets against 171 (Supplementary Table S6). How much search time is worthwhile depends on the cost of an evaluation. `FMQA-SA` spends about 0.5 s on search per evaluation, comparable to the evaluations in these models, while `RBFOpt` spends 17 to 98 ms in median but more as observations accumulate. `MISO` spends 52 ms to 1.86 s per evaluation, and on `Dynamic` its search per evaluation exceeds the median evaluation cost of 0.84 s; `NOMAD` spends 3 to 97 ms (Supplementary Table S12). When each evaluation requires a longer optimization or a process simulation, this overhead becomes small, and the number of subproblem solves becomes the main cost. D: This is to be expected; by preprocessing, you do not solve problems. But mention which solvers stall and which keep growing with more iterations; also mention if there is a tradeoff search time and solution quality. A: Added the Dynamic budget comparison showing where target-reaching rates improve or plateau and how completion costs change.

Because the initial configurations change with the seed and the surrogate-based methods are randomized, repeated runs can return different solutions, and the spread of these solutions differs between methods. `RBFOpt` returns at most three distinct final objectives across the ten seeds of each `CSTR`, `Small Batch`, and `Dynamic` condition; on `CSTR10` through `CSTR30`, its runs either reach the target or stop 8.1 to 12.5% above it. `MISO` and `NOMAD` are similarly consistent, with at most five and three distinct final objectives per condition,

and the MISO misses on CSTR10 and CSTR15 lie 6.0 to 10.6% above the reference. BAM reaches the exact target in only 18 of the 60 CSTR runs, but all of its final objectives lie within 0.7% of the reference. FMQA-SA and uniform sampling spread more widely, with up to ten distinct final objectives in one condition and gaps of up to 17%. With a relative tolerance of 1%, BAM would succeed in all 60 CSTR runs, compared with 45 for RBFOpt, 43 for MISO, 21 for NOMAD, and 19 for D-SDA (Supplementary Table S11), so the strict target separates exact attainment from near-optimal solutions that may suffice in a design study.

The comparison with enumeration and conventional GDP methods indicates when each approach is preferable. For small models such as Small Batch and the Grid functions, enumeration or a direct reformulation finds the optimum in seconds or less. As the configuration space grows, logic-based methods that solve one subproblem per admissible configuration and direct reformulations that reach the time limit become expensive, while surrogate search keeps its number of subproblem solves near the budget, as on CSTR30. Dynamic lies between these cases: RIC reaches its target in 149.39 s after solving all 37 admissible subproblems, and RBFOpt and MISO reach it in every run at $B = 12$ in medians of 12.29 and 11.80 s. D: This is the usual AI warning about scalability; rather, say which cases benefited from the conventional GDP solution models than the integration with DFOs, and in the conclusion you mention this as recommendations for future instances A: Replaced the generic scalability warning with the Small Batch and CSTR30 comparisons and added their implications to the conclusions.

The Lomustine case shows that the configuration search carries over when a simulator defines each subproblem and conventional GDP methods do not apply. Its ranking agrees with the CSTR and Grid results: the two radial-basis-function methods reach the reference most often, NOMAD stops early, and the size of the D-SDA neighborhood limits local descent on small budgets. Because each evaluation costs minutes, the search overhead that slows MISO on Dynamic is negligible here.

These conclusions rest on configuration spaces of 25 to 900 configurations, on NLP responses obtained with one initialization per model, and on completion times that combine recorded search paths with one cost measurement per configuration. Because the CSTR budgets grow with N, the CSTR series does not isolate the effect of size. With ten seeds per condition, the per-condition differences among NOMAD, MISO, RBFOpt, and D-SDA are not significant after correcting for the 70 exploratory tests, so rankings within a single condition are descriptive. The NOMAD and MISO search times were measured in a later campaign on a shared server, so small timing differences between them and the other methods should not be overinterpreted.

5 Conclusions

This study compared black-box methods for GDP configuration search on an existing external integer representation with local descent, logic-based and direct GDP methods, and full enumeration. The experiments lead to four conclusions.

First, surrogate search reaches near-optimal configurations with a small fraction of the NLP subproblems that logic-based methods solve. In these models, RIC and LOA solved one subproblem for each admissible configuration, whereas MISO and RBFOpt reached the CSTR30 target in ten and nine of ten runs with medians of 29.5 and 31.5 subproblems, 6 to 7% of full enumeration.

Second, the benefit over local descent depends on the landscape. On CSTR, most D-SDA descents end at one local minimum whose gap to the reference grows with the network size, and restarts add a single target; the direct search of NOMAD ends at the same configuration in 32 of its 33 misses on the larger networks. RBFOpt and MISO, which interpolate all observations with radial basis functions, escape it in 45 and 43 of 60 runs. On the two smallest design models, D-SDA and RBFOpt perform comparably, and MISO reached all ten Column targets.

Third, the stochastic methods differ in how their solutions spread across runs. RBFOpt and MISO either reached the target or stopped at a local solution 6.0 to 12.5% above it on the larger CSTR instances, whereas BAM rarely reached the exact target but always ended within 0.7% of it. Reporting only exact target attainment can therefore hide reliable near-optimal performance. D: Sure, this is obvious; any deeper insights? How about the fact that they are stochastic? That you would get different solutions when running them separately; were they spread out? A: Added the contrasting seed-level results for Dynamic and Column to describe consistency in target discovery and variation in final solution quality.

Fourth, more search per evaluation can buy targets, but its value depends on the cost of an evaluation.

MISO reached the most targets overall, 181 of 210 on Grid at $B = 100$ and 120 of 140 in the process conditions, against 171 and 114 for RBFOpt, but on Dynamic at $B = 50$ its model refits made it three times slower than RBFOpt. NOMAD and D-SDA, in contrast, stop at local optima and leave budget unused.

For future GDP instances, these results suggest enumerating the configurations or solving a direct Big-M or Hull reformulation when the model is small, as for Grid-GDP and Small Batch. When the configuration space is too large to enumerate and contains large basins of local minima, as for the larger CSTR networks, surrogate-based configuration search with radial basis functions reaches good solutions with a small fraction of the subproblem solves; MISO is preferable when evaluations are expensive relative to its search overhead, and RBFOpt when they are cheap. The wall-time advantage of surrogate search grows with the cost of each evaluation, and because it requires only objective values, it also applies when configurations are evaluated by simulation and explicit subproblem equations are unavailable, as in the Lomustine flowsheet, where RBFOpt and MISO found the best of 36 simulated configurations in every run with 20 evaluations. Testing these expectations on larger configuration spaces and on live simulation-based subproblems is the next step. D: how is this a conclusion? What did you learn from all these experiments? A: Revised the paragraph to explain what the Small Batch and CSTR30 comparisons teach us and identify simulation-based subproblems as a future application.

6 Acknowledgement

The authors acknowledge and express sincere gratitude to the Davidson School of Chemical Engineering at Purdue University for providing a supportive research environment and funding.

Data and code availability

The code and data are available in the Black-Box-for-GDP repository (SECQUOIA, 2026a). Supplementary Section S7 identifies the archived inputs, including the CSTR updates, Column logical-domain runs, and conventional GDP comparisons.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

The authors used ChatGPT and Codex (OpenAI), including the GPT-6 Astra model, to assist with drafting, organizing, and revising the manuscript and developing analysis and plotting code. The authors also used Claude Code (Anthropic), with the Claude Opus 5.5 model, to review the manuscript, revise the Discussion and Conclusions, and develop analysis, plotting, and experiment code. D: which models? A: Specified GPT-6 Astra and described AI-assisted code development and verification in the Methods. The resulting text and code were checked and revised by the authors, who remain responsible for the article's content.

References

Abramson, M.A., Audet, C., Chrissis, J.W., Walston, J.G., 2009. Mesh adaptive direct search algorithms for mixed variable optimization. Optim. Lett. 3, 35–47. doi:10.1007/s11590-008-0089-2.

Amestoy, P.R., Duff, I.S., Koster, J., L'Excellent, J.Y., 2001. A fully asynchronous multifrontal solver using distributed dynamic scheduling. SIAM Journal on Matrix Analysis and Applications 23, 15–41. doi:10.1137/S0895479899358194.

Audet, C., Le Digabel, S., Rochon Montplaisir, V., Tribes, C., 2022. Algorithm 1027: NOMAD version 4: Nonlinear optimization with the MADS algorithm. ACM Trans. Math. Softw. 48, 35:1–35:22. doi:10.1145/3544489.

Baptista, R., Poloczek, M., 2018. Bayesian optimization of combinatorial structures, in: Proceedings of the 35th International Conference on Machine Learning, pp. 462–471.

Barhate, Y., Laky, D.J., Casas-Orozco, D., Reklaitis, G.V., Nagy, Z.K., 2025. Hybrid rule-based and optimization-driven framework for the synthesis of end-to-end optimal pharmaceutical processes. AIChE Journal , e18888doi:10.1002/aic.18888.

Bergamini, M.L., Aguirre, P., Grossmann, I., 2005. Logic-based outer approximation for globally optimal synthesis of process networks. Comput. Chem. Eng. 29, 1914–1933. doi:10.1016/j.compchemeng.2005.04.003.

Bolusani, S., Besançon, M., Bestuzheva, K., Chmiela, A., Dionísio, J., Donkiewicz, T., van Doornmalen, J., Eifler, L., Ghannam, M., Gleixner, A., Graczyk, C., Halbig, K., Hedtke, I., Hoen, A., Hojny, C., van der Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Manns, J., Mexi, G., Mühmer, E., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Weninger, D., Xu, L., 2024. The SCIP Optimization Suite 9.0. ZIB Report 24-02-29. Zuse Institute Berlin. URL: https://nbn-resolving.org/urn:nbn:de:0297-zib-95528.

Bonami, P., Biegler, L.T., Conn, A.R., Cornuéjols, G., Grossmann, I.E., Laird, C.D., Lee, J., Lodi, A., Margot, F., Sawaya, N., Wächter, A., 2008. An algorithmic framework for convex mixed integer nonlinear programs. Discrete Optimization 5, 186–204. doi:10.1016/j.disopt.2006.10.011.

Bynum, M.L., Hackebeil, G.A., Hart, W.E., Laird, C.D., Nicholson, B.L., Siirola, J.D., Watson, J.P., Woodruff, D.L., 2021. Pyomo: Optimization Modeling in Python. volume 67 of *Springer Optimization and Its Applications*. 3 ed., Springer. doi:10.1007/978-3-030-68928-5.

Casas-Orozco, D., Laky, D., Wang, V., Abdi, M., Feng, X., Wood, E., Laird, C., Reklaitis, G.V., Nagy, Z.K., 2021. PharmaPy: An object-oriented tool for the development of hybrid pharmaceutical flowsheets. Computers & Chemical Engineering 153, 107408. doi:10.1016/j.compchemeng.2021.107408.

Casas-Orozco, D., Laky, D.J., Wang, V., Abdi, M., Feng, X., Wood, E., Reklaitis, G.V., Nagy, Z.K., 2023. Techno-economic analysis of dynamic, end-to-end optimal pharmaceutical campaign manufacturing using PharmaPy. AIChE Journal 69, e18142. doi:10.1002/aic.18142.

Chen, Q., Johnson, E.S., Bernal, D.E., Valentin, R., Kale, S., Bates, J., Siirola, J.D., Grossmann, I.E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. Optim. Eng. 23, 607–642. doi:10.1007/s11081-021-09601-7.

Costa, A., Nannicini, G., 2018. RBFOpt: an open-source library for black-box optimization with costly function evaluations. Math. Program. Comput. 10, 597–629. doi:10.1007/s12532-018-0144-7.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60, 2211–2227. doi:10.1002/aic.14418.

D-Wave Systems, n.d. dwave-samplers: Classical algorithms for solving binary quadratic models. URL: https://github.com/dwavesystems/dwave-samplers/releases/tag/1.6.0. Version 1.6.0; classical CPU simulated annealing; accessed October 15, 2026.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Math. Program. 91, 201–213. doi:10.1007/s101070100263.

GAMS Development Corporation, 2025. General Algebraic Modeling System (GAMS). URL: https://www.gams.com/. release 52.2.0, Fairfax, VA, USA.

Ghouse, J.H., Chen, Q., Zamarripa, M.A., Lee, A., Burgard, A.P., Grossmann, I.E., Miller, D.C., 2018. A comparative study between gdp and nlp formulations for conceptual design of distillation columns, in: Computer Aided Chemical Engineering, pp. 865–870. doi:10.1016/B978-0-444-64241-7.50139-7.

Grossmann, I.E., Lee, S., 2003. Generalized convex disjunctive programming: Nonlinear convex hull relaxation. Comput. Optim. Appl. 26, 83–100. doi:10.1023/A:1025154322278.

Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/1412.6980.

Kirkpatrick, S., Gelatt, Jr., C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Kitai, K., Guo, J., Ju, S., Tanaka, S., Tsuda, K., Shiomi, J., Tamura, R., 2020. Designing metamaterials with quantum annealing and factorization machines. Phys. Rev. Res. 2, 013319. doi:10.1103/PhysRevResearch.2.013319.

Kocis, G.R., Grossmann, I.E., 1988. Global optimization of nonconvex mixed-integer nonlinear programming (minlp) problems in process synthesis. Industrial & Engineering Chemistry Research 27, 1407–1421. doi:10.1021/ie00080a013.

Laky, D., Casas-Orozco, D., 2022. Superstructure synthesis and simulation-optimization of pharmaceutical manufacturing processes using PharmaPy. AIChE Annual Meeting, Phoenix, AZ, paper 183b.

Larson, J., Menickelly, M., Wild, S.M., 2019. Derivative-free optimization methods. Acta Numerica 28, 287–404. doi:10.1017/S0962492919000060.

Liñán, D.A., Bernal, D.E., Ricardez-Sandoval, L.A., Gómez, J.M., 2020. Optimal design of superstructures for placing units and streams with multiple and ordered available locations. part i: A new mathematical framework. Computers & Chemical Engineering 137, 106794. doi:10.1016/j.compchemeng.2020.106794.

Ma, K., Rios, L.M., Bhosekar, A., Sahinidis, N.V., Rajagopalan, S., 2023. Branch-and-Model: a derivative-free global optimization algorithm. Comput. Optim. Appl. 85, 337–367. doi:10.1007/s10589-023-00466-3.

Moré, J.J., Wild, S.M., 2009. Benchmarking derivative-free optimization algorithms. SIAM J. Optim. 20, 172–191. doi:10.1137/080724083.

Müller, J., 2016. MISO: mixed-integer surrogate optimization framework. Optim. Eng. 17, 177–203. doi:10.1007/s11081-015-9281-2.

Nannicini, G., 2021. On the implementation of a global optimization method for mixed-variable problems. Open J. Math. Optim. 2, 1–25. doi:10.5802/ojmo.3.

Oh, C., Tomczak, J.M., Gavves, E., Welling, M., 2019. Combinatorial Bayesian optimization using the graph Cartesian product, in: Advances in Neural Information Processing Systems.

Ovalle, D., Liñán, D.A., Lee, A., Gómez, J.M., Ricardez-Sandoval, L., Grossmann, I.E., Bernal Neira, D.E., 2025. Logic-Based Discrete-Steepest Descent: A solution method for process synthesis Generalized Disjunctive Programs. Comput. Chem. Eng. 195, 108993. doi:10.1016/j.compchemeng.2024.108993.

Park, Y., Bernal Neira, D.E., 2026. A computational study of Ising-based solvers for discrete landscape exploration in process optimization. AIChE Journal , e70656doi:10.1002/aic.70656.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, pp. 8024–8035.

Peng, Z., Lee, A., Bernal Neira, D.E., 2024. Addressing discrete dynamic optimization via a logic-based discrete-steepest descent algorithm, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), IEEE. pp. 1664–1669. doi:10.1109/CDC56724.2024.10886477.

Ploskas, N., Sahinidis, N.V., 2022. Review and comparison of algorithms and software for mixed-integer derivative-free optimization. J. Glob. Optim. 82, 433–462. doi:10.1007/s10898-021-01085-0.

Pyomo Development Team, n.d. GDPopt logic-based solver. URL: https://pyomo.readthedocs.io/en/6.10.0/explanation/solvers/gdpopt.html. Version 6.10.0; accessed October 20, 2026.

Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. Comput. Chem. Eng. 18, 563–578. doi:10.1016/0098-1354(93)E0010-7.

Regis, R.G., Shoemaker, C.A., 2007. A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. 19, 497–509. doi:10.1287/ijoc.1060.0182.

Rendle, S., 2010. Factorization machines, in: 2010 IEEE International Conference on Data Mining, pp. 995–1000. doi:10.1109/ICDM.2010.127.

Sahinidis, N.V., 1996. BARON: A general purpose global optimization software package. J. Glob. Optim. 8, 201–205. doi:10.1007/BF00138693.

SECQUOIA, 2026a. Black-Box-for-GDP: benchmark code, experimental data, and analysis. GitHub. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/49d4bcbeb10fe49b74fe2333d27829046e93bf03. commit 49d4bcbe; accessed 28 October 2026.

SECQUOIA, 2026b. pd_ising: Lomustine superstructure PharmaPy simulation workflow and data. URL: https://github.com/SECQUOIA/pd_ising/tree/633cf923f07abba88970fa1fa816c4d06769751e/ds-mfg. pull request 28; commit 633cf923f07abba88970fa1fa816c4d06769751e; accessed October 31, 2026.

SECQUOIA, n.d. bbo_via_fmqa: Black-box optimization via factorization machines and quantum annealing. URL: https://github.com/SECQUOIA/bbo_via_fmqa/tree/eb3a20d4b5a3773d17ff31d72a35d51f58e84e56. QHD_Data branch; commit eb3a20d4b5a3773d17ff31d72a35d51f58e84e56; upstream PyTorch FM and BQM code; accessed October 19, 2026.

The Optimization Firm, 2026. BAM python interface documentation: bam-solver 5.0.0. URL: https://pypi.org/project/bam-solver/5.0.0/. Version 5.0.0, released September 11, 2026; executed engine 2026.9.10; accessed October 20, 2026.

Türkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. Comput. Chem. Eng. 20, 959–978. doi:10.1016/0098-1354(95)00219-7.

Wächter, A., Biegler, L.T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming 106, 25–57. doi:10.1007/s10107-004-0559-y.

Wu Group, n.d. Quantum Hamiltonian descent: 2D test functions. https://jiaqileng.github.io/quantum-hamiltonian-descent/nonconvex2d.html. Source collection for the 21 adapted finite grids used here; accessed September 29, 2026.

Supplementary Material: Benchmark Definitions, Execution Protocol, and Uncertainty

The Grid-GDP functions are adapted from the Quantum Hamiltonian Descent collection; each instance is defined by its implemented expression and domain rather than its conventional name (SECQUOIA, 2026a; Wu Group, n.d.).

S1 Coordinate domains and normalization

The coordinate mapping and objective normalization follow Section 3.3 of the main paper. Both coordinate endpoints are included, and L and U are the minimum and maximum over the 961 lattice points. All trigonometric arguments are in radians. Full-precision normalization constants and target coordinates are archived in `grid_inventory.json` (Section S7).

Table S1: Grid-GDP benchmark domains and target-set sizes. Each problem uses a 31×31 lattice with 961 configurations and 62 Boolean indicators. n_{target} counts lattice points with normalized objective $F \leq 10^{-5}$.

Function	x domain	y domain	n_{target}
Ackley	$[-5, 5]$	$[-5, 5]$	1
Alpine 1	$[0, 10]$	$[0, 10]$	1
Alpine 2	$[0, 1000]$	$[0, 1000]$	1
Bohachevsky 2	$[-100, 100]$	$[-100, 100]$	1
Three-hump camel	$[-5, 5]$	$[-5, 5]$	1
Csendes (squared variant)	$[-1, 1]$	$[-1, 1]$	36
Deflected corrugated spring	$[0, 10]$	$[0, 10]$	1
Drop-Wave	$[-5.12, 5.12]$	$[-5.12, 5.12]$	1
Easom	$[-10, 10]$	$[-10, 10]$	1
Griewank	$[-600, 600]$	$[-600, 600]$	1
Hölder table	$[-10, 10]$	$[-10, 10]$	4
Hosaki	$[0, 5]$	$[0, 6]$	1
Lévy	$[-10, 10]$	$[-10, 10]$	2
Lévy 13	$[-10, 10]$	$[-10, 10]$	4
Michalewicz	$[0, \pi]$	$[0, \pi]$	1
Rastrigin	$[-5.12, 5.12]$	$[-5.12, 5.12]$	1
Rosenbrock	$[-2, 2]$	$[-1, 3]$	2
Shubert (variant)	$[0, 10]$	$[0, 10]$	1
Styblinski–Tang	$[-5, 5]$	$[-5, 5]$	1
Sum of squares	$[-10, 10]$	$[-10, 10]$	1
Xin-She Yang 3 (variant)	$[-20, 20]$	$[-20, 20]$	870

S2 Grid-GDP objective functions

The implemented functions below use the domains in Table S1 (SECQUOIA, 2026a).

1. Ackley

$$\begin{aligned} f(x, y) = & -20 \exp\left(-0.2\sqrt{\frac{x^2 + y^2}{2}}\right) \\ & - \exp\left(\frac{\cos(2\pi x) + \cos(2\pi y)}{2}\right) + 20 + e. \end{aligned} \tag{S1}$$

2. Alpine 1

$$f(x,y) = |x\sin x + 0.1x| + |y\sin y + 0.1y|. \tag{S2}$$

3. Alpine 2

$$f(x,y) = -\sqrt{xy}\,\sin x \sin y. \tag{S3}$$

4. Bohachevsky 2

$$f(x,y) = x^2 + 2y^2 - 0.3\cos(3\pi x)\cos(4\pi y) + 0.3. \tag{S4}$$

5. Three-hump camel

$$f(x,y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2. \tag{S5}$$

6. Csendes (squared variant)

$$f(x,y) = c(x) + c(y), \qquad c(t) = \begin{cases} t^6\big[2 + \sin(1/t)\big]^2, & t \neq 0, \\ 0, & t = 0. \end{cases} \tag{S6}$$

The GDP uses branch-fixed auxiliaries $r_t = 1/t$ for $t \neq 0$ and $r_t = 0$ otherwise, avoiding division by zero without an epsilon shift.

7. Deflected corrugated spring

With $r = \sqrt{(x-5)^2 + (y-5)^2}$,

$$f(x,y) = 0.1r^2 - \cos(5r). \tag{S7}$$

8. Drop-Wave

$$f(x,y) = -\frac{1 + \cos\Big(12\sqrt{x^2+y^2}\Big)}{0.5(x^2+y^2) + 2}. \tag{S8}$$

9. Easom

$$f(x,y) = -\cos x \cos y\, \exp\big(-[(x-\pi)^2 + (y-\pi)^2]\big)\,. \tag{S9}$$

10. Griewank

$$f(x,y) = \frac{x^2+y^2}{4000} - \cos x \cos\left(\frac{y}{\sqrt{2}}\right) + 1. \tag{S10}$$

11. Hölder table

$$f(x,y) = -\left|\sin x \cos y\, \exp\left(\left|1 - \frac{\sqrt{x^2+y^2}}{\pi}\right|\right)\right|. \tag{S11}$$

12. Hosaki

$$f(x,y) = \left(1 - 8x + 7x^2 - \frac{7}{3}x^3 + \frac{1}{4}x^4\right) y^2 \exp(-y). \tag{S12}$$

13. Lévy

Define $w_x = 1 + (x-1)/4$ and $w_y = 1 + (y-1)/4$. Then

$$\begin{aligned} f(x,y) = \; & \sin^2(\pi w_x) \\ & + (w_x - 1)^2\big[1 + 10\sin^2(\pi w_x + 1)\big] \\ & + (w_y - 1)^2\big[1 + \sin^2(2\pi w_y)\big]. \end{aligned} \tag{S13}$$

14. Lévy 13

$$\begin{aligned} f(x,y) = \; & \sin^2(3\pi x) + (x-1)^2\big[1 + \sin^2(3\pi y)\big] \\ & + (y-1)^2\big[1 + \sin^2(2\pi y)\big]. \end{aligned} \tag{S14}$$

15. Michalewicz

$$f(x,y) = -\sin x \, \sin^{20}\!\left(\frac{x^2}{\pi}\right) - \sin y \, \sin^{20}\!\left(\frac{2y^2}{\pi}\right). \tag{S15}$$

16. Rastrigin

$$f(x,y) = x^2 - 10\cos(2\pi x) + y^2 - 10\cos(2\pi y) + 20. \tag{S16}$$

17. Rosenbrock

$$f(x,y) = (1-x)^2 + 100(y - x^2)^2. \tag{S17}$$

18. Shubert (variant)

$$\begin{aligned} f(x,y) = \; & \big[\cos(2x+1) + 2\cos(3x+2) + 3\cos(4x+3)\big] \\ & \times \big[\cos(2y+1) + \cos(y+2)\big]. \end{aligned} \tag{S18}$$

19. Styblinski–Tang

$$f(x,y) = \frac{1}{78}\,\frac{1}{2}\left(x^4 - 16x^2 + 5x + y^4 - 16y^2 + 5y\right). \tag{S19}$$

20. Sum of squares

$$f(x,y) = x^2 + 2y^2. \tag{S20}$$

21. Xin-She Yang 3 (variant)

$$f(x,y) = \exp\!\left(-\left[\left(\frac{x}{15}\right)^6 + y^6\right]\right) + 2\exp\!\big(-(x^2+y^2)\big)\cos^2 x \cos^2 y. \tag{S21}$$

S3 Executable protocol and implementation settings

Python and NumPy use the job seed, and `RBFOpt` also receives `rand_seed`. `BAM` exposes no separate native random-seed option: its process wrapper seeds Python/NumPy, while the Grid seed identifies the common initial design. `FMQA-SA` uses three NumPy `SeedSequence(seed)` streams for FM initialization, annealing, and uniform unseen-point fallback; each training or annealing invocation draws a seed from its stream. `PyTorch` uses deterministic CPU operations and one thread.

Grid `D-SDA` enumerates L_∞ neighbors lexicographically from $(-1, 0, 1)$, excluding the zero direction and out-of-bounds points; ties select the first point.

The RBFOpt options are rbf=auto, global_search_method=genetic, do_init_strategy=False, ga_base_population_size=400, and ga_num_generations=20.

BAM receives x0=None and max_evals=B-3; its local models use ALAMO/BARON, whose internal versions are not reported by the package.

The FMQA-SA time limit is checked between operations and through training/sampling callbacks. Each FM uses identity activation, zero bias, and normal initial weights with mean zero and standard deviation 0.01. Training uses Adam (Kingma and Ba, 2015), 1,000 full-batch epochs, mean squared error, and learning rate 0.01. Coordinate ξ_j uses $\lfloor \log_2 u_j \rfloor + 1$ unsigned bits, where $u_j > 0$ is its upper bound; QUBO coefficients are divided by the largest absolute entry of the quadratic matrix. D-Wave's SimulatedAnnealingSampler in dwave-samplers 1.6.0 uses 100 reads, 1,000 sweeps, and a geometric inverse-temperature schedule (D-Wave Systems, n.d.). All process workers and the Grid FMQA-SA worker have a 2,100 s watchdog. The Grid RBFOpt/BAM trace watchdogs are disabled (max_run_seconds=0); archived Grid D-SDA uses budget, target, or local-search termination without an additional method-clock limit.

S3.1 Versions and independently timed process evaluations

The recorded Python version is 3.11.16; other versions are given in the main paper (Section 3.5).

The conventional GDP process models retain their archived initializations, including constructor defaults for CSTR30; the updated DFO evaluation table is identified separately in Section S7.

The acceptance rule in gdp_models/evaluation.py requires all of the following: a finite objective; termination in {optimal, locallyOptimal, globallyOptimal}; solver status other than error or aborted; interface status SOLVESTAT=1 and MODELSTAT$\in \{1, 2\}$; and a finite maximum primal violation no greater than 10^{-5}. Time-limit, iteration-limit, evaluation-limit, and error outcomes are not accepted, even if a finite feasible candidate is available.

The primal check uses unscaled violations of original constraints and initialized-variable bounds, including constraint rows deactivated during preprocessing. Nonfinite objectives, constraint bodies, or initialized-variable values preclude acceptance; unused uninitialized variables are skipped. Preprocessing rejection or worker failure supplies no accepted objective.

The CSTR initializer cstr_box_ipopt.py:initialize_feasible runs after preprocessing and checks the mass-balance/zero-recycle start at 10^{-8} without changing constraints or bounds. All process evaluations use cold starts; NLP solutions are not transferred between configurations. Objectives, acceptance status, and NLP point costs are linked to the same recorded attempt by model, CSTR size, and coordinate, without successful-retry selection or cross-size time substitution.

S3.2 Observed proposal and initialization behavior

Table S2 pools FMQA-SA proposals across budgets and seeds, excluding the shared initialization and including terminal callbacks. Fallback supplies 52.89% of process proposals; the uniform comparison evaluates the complete procedure, without isolating the contributions of learning, annealing, and fallback.

Table S2: Recorded FMQA-SA uniform fallback by benchmark family.

Family	Runs	Proposals	Fallbacks	Rate
Grid	1,050	51,450	9,522	18.51%
CSTR	60	1,350	369	27.33%
Column	10	100	86	86.00%
Dynamic	60	1,460	1,063	72.81%
Small Batch	10	70	58	82.86%
Process total	140	2,980	1,576	52.89%

S4 Process model scale, units, and discrete decisions

Table S3 counts the original GDP before reformulation or presolve, including all active disjunct branches and fixed continuous variables. `Dynamic` is discretized with 30 finite elements per stage and three Radau points. Binary counts refer to the indicator variables already constructed by the model, before Boolean-to-binary conversion. All nine models have zero additional nonbinary integer variables.

Table S3: Original process-model sizes. n_c: continuous variables; n_b: binary variables; m_a: algebraic constraints; m_l: logical constraints; n_d: disjunctions.

Model	n_c	n_b	m_a	m_l	n_d
`CSTR5`	56	20	100	12	10
`CSTR10`	106	40	195	22	20
`CSTR15`	156	60	290	32	30
`CSTR20`	206	80	385	42	40
`CSTR25`	256	100	480	52	50
`CSTR30`	306	120	575	62	60
`Column`	446	30	637	16	15
`Small Batch`	19	18	34	3	9
`Dynamic`	1647	27	3276	31	9

The `Column` objective is $10^3(Q_c + Q_b) + 10^3 n_{\text{active}}$, the source model's weighted cost proxy rather than a newly estimated annualized cost. The `Dynamic` objective is $-\int_0^9 x(t)^2\,dt$.

S5 Lomustine simulation-based benchmark

The `Lomustine` configurations and responses come from the corrected-model tables of SECQUOIA (2026b), generated with PharmaPy at commit 3194efd. For each superstructure option, the source ran `PyNomad` 4.6.0 from a stored initial point on the normalized operating inputs with 60 simulations and stored the best objective, the constraint values, the equipment sizes, and the wall time of the inner optimization. The inner search accepts a point when the solubility path constraint holds. The heptane mass-fraction specification of the source is not enforced, because the fixed three-hour vaporizer operation cannot reach it within the pressure bounds. Table S4 lists the 36 two-reactor configurations; the four single-reactor semibatch options of the source do not fit the three external variables and are excluded. The configurations with a batch crystallizer have the lowest objective for every reactor pair, and the choice of the first reactor changes the objective by up to a factor of 2.9. The vendored table, case module, run records, and analysis are listed in Section S7.

Table S4: `Lomustine`: objective of every configuration and, in parentheses, the wall time of its simulation-based optimization. Rows give the first (R01) and second (R02) reactor, and columns the crystallizer. The bold entry is the reference configuration.

R01	R02	batch	1 MSMPR	2 MSMPR	3 MSMPR
batch	batch	-170.68 (91 s)	-131.70 (310 s)	-146.86 (538 s)	-148.92 (835 s)
batch	CSTR	-230.52 (146 s)	-175.25 (348 s)	-175.27 (650 s)	-166.13 (970 s)
batch	PFR	-268.46 (159 s)	-209.36 (348 s)	-213.29 (651 s)	-202.26 (975 s)
CSTR	batch	-483.85 (152 s)	-360.16 (331 s)	-365.91 (656 s)	-345.28 (936 s)
CSTR	CSTR	-331.90 (159 s)	-244.39 (367 s)	-245.34 (660 s)	-239.50 (1019 s)
CSTR	PFR	-429.81 (206 s)	-315.22 (408 s)	-321.46 (717 s)	-304.62 (1017 s)
PFR	batch	**-487.84** (208 s)	-362.45 (347 s)	-361.55 (655 s)	-348.06 (886 s)
PFR	CSTR	-351.71 (183 s)	-249.82 (388 s)	-259.47 (659 s)	-244.65 (983 s)
PFR	PFR	-433.41 (211 s)	-318.34 (393 s)	-324.18 (629 s)	-306.21 (907 s)

S6 Completion costs

Table S5 reports completion costs for all 14 process conditions, together with the cost of evaluating every configuration of each DFO domain once. Table S6 reports the Grid-GDP costs at $B = 100$. `MISO` reaches the most Grid targets, but its median search time of 32.70 s is about four times that of `RBFOpt`, whereas `D-SDA` and `NOMAD` stop after medians of 37 and 31.5 evaluations and have the lowest median completion times, 18.38 and 18.50 s (`NOMAD` over its 189 timed runs).

Per-function Grid results at $B = 100$ and complete seed-level outcomes are available in the analysis files linked in Section S7.

Table S5: Median reconstructed full-run completion costs for all ten paired seeds per condition, including target misses, and the cost of full enumeration, in seconds.

Case	B	D-SDA	NOMAD	RBFOpt	MISO	BAM	FMQA-SA	Uniform	Enumeration
Column	13	35.44	31.93	54.60	31.98	27.50	41.82	21.69	137.24
CSTR5	8	10.67	10.71	10.72	11.33	10.67	13.92	9.44	30.27
CSTR10	15	17.61	21.25	21.79	21.87	18.54	27.07	17.75	122.06
CSTR15	22	25.68	33.92	37.59	39.79	27.67	42.82	27.84	298.42
CSTR20	29	32.31	38.17	45.03	56.03	39.37	59.10	40.12	553.38
CSTR25	36	45.49	56.56	62.29	78.56	50.47	74.70	51.17	899.39
CSTR30	43	47.17	71.23	101.96	107.33	115.41	101.19	65.20	1719.00
Small Batch	10	4.23	4.97	5.12	5.62	4.23	9.09	3.49	10.18
Dynamic	12	12.22	12.36	12.29	11.80	9.95	14.47	7.80	47.25
Dynamic	15	15.32	15.48	16.10	15.44	11.53	18.39	8.71	47.25
Dynamic	20	20.02	20.85	20.54	42.42	14.07	25.23	12.27	47.25
Dynamic	30	20.02	22.00	34.19	106.24	21.91	36.24	19.38	47.25
Dynamic	37	20.02	22.00	42.42	111.61	24.99	46.05	24.38	47.25
Dynamic	50	20.02	21.99	65.14	195.36	33.98	61.31	33.12	47.25

Table S6: Grid-GDP results at $B = 100$, with 210 runs per method (21 functions and ten seeds). Target hits reach the normalized reference zero within 10^{-5} (Equation (8)). Times are Q50 (median) [Q25, Q75] in seconds across all runs, including target misses, using linearly interpolated quartiles. Summed configuration-evaluation cost includes the separately measured cost of every evaluation in the full run, including repeated configurations. Search time excludes lookup processing; adding it to the summed configuration-evaluation cost gives estimated full-run completion time (Equation (10)). Component quartiles need not sum to completion quartiles. [a] NOMAD times cover the 189 runs with recorded times; the 21 runs that crashed on every attempt have none, but their target hits are counted (Section S11).

Method	Target hits	Full-run completion (s)	Summed configuration-evaluation cost (s)	Search time (s)
D-SDA	101/210	18.38 [12.42, 37.06]	18.38 [12.41, 37.05]	0.0024 [0.0023, 0.0026]
NOMAD[a]	126/210	18.50 [13.46, 35.00]	15.20 [11.09, 30.82]	2.04 [1.79, 2.28]
RBFOpt	171/210	70.81 [54.83, 93.09]	60.84 [44.31, 82.15]	7.58 [4.12, 14.82]
MISO	181/210	94.82 [79.22, 116.05]	63.21 [44.03, 82.37]	32.70 [25.19, 39.38]
BAM	147/210	64.42 [48.74, 81.67]	63.97 [48.34, 81.22]	0.43 [0.37, 0.51]
FMQA-SA	129/210	124.13 [105.57, 139.26]	62.34 [43.98, 77.52]	62.04 [61.58, 62.53]
Uniform	45/210	60.51 [43.33, 78.60]	60.51 [43.33, 78.59]	0.0026 [0.0026, 0.0027]
BAM-prefix control	124/210	61.81 [40.96, 78.33]	61.80 [40.95, 78.33]	0.0026 [0.0026, 0.0027]

S7 Data sources and reproducibility

The publication snapshot contains the source code, experimental data, analysis results, and reproduction instructions for the DFO and conventional GDP comparisons. The updated GDPopt analysis and execution records contain the 63 Grid and 27 process runs configured with `SCIP` for MIP and `IPOPT` for NLP subproblems. The three presolve-disabled LOA diagnostic runs are excluded. The same records contain the paired comparison with the earlier configuration that used `SCIP` for both MIP and NLP subproblems, in which GLOA returned the same `CSTR` objectives within a relative difference of 3×10^{-5}. Seed spread, tolerance sensitivity, search overhead, and `GDPopt` enumeration counts are generated by `review_comparison/revision_diagnostics.py` from the retained run records and execution logs. The `D-SDA` rerun, including the process restart variants, is `experiments/dsda_no_revisit/run.py`, which replays the sealed lookup package of the adopted `D-SDA` rerun. The `NOMAD` and `MISO` runs were executed at source revision c4d7db6; the returned records contain the original archive, its SHA-256 checksum, and hash-checked analysis tables of every run. All `D-SDA` runs, including the restart variants, were rerun at source revision c186408, where no configuration is evaluated twice; with re-evaluation switched back on, the same code reproduces all 1,190 published `D-SDA` runs exactly, and the returned records hold every run. The `Lomustine` case is `experiments/lomustine_simulation`, which vendors the corrected-model table of SECQUOIA (2026b) and runs the seven methods through the harness in `experiments/catalytic_distillation_live`; the run records and analysis are in `results/lomustine_simulation_20261031`. The seven-method tables and figures are generated from these tables and the published records by `review_comparison/seven_methods.py` and the `review_comparison/joint_process_*.py` scripts, without new solver calls.

S8 Feasibility and bound consistency of conventional GDP results

Of the 45 conventional GDP runs in Table S15, 21 have flagged bounds (Table S7) relative to the checked solutions in Table S8. D: we agreed that we are not calling algebraic the conventional methods for GDP A: Replaced algebraic with conventional GDP for the compared methods throughout the supplement, including the section heading. These results combine the 27 new `SCIP`–`IPOPT` GDPopt runs with the 18 retained direct `SCIP` runs. Unrounded values, residuals, and source records are linked in Section S7.

For each minimization model, let f_w be the recorded objective of its best independently checked saved feasible solution, and let L denote a reported lower bound. We apply the same rule to all five methods:

$$\tau_w = 10^{-5} \max\{1, |f_w|\}, \qquad \Delta = L - f_w, \qquad \rho = \Delta/\tau_w, \qquad \dagger \iff \rho > 1. \tag{S22}$$

Missing lower bounds remain missing, not zero.

Saved vectors are loaded into the original GDP to reevaluate objectives and check active constraints, variable bounds, integrality, disjunctions, and logical validity without optimization. Violations are unscaled original-equation residuals, not reformulation residuals. All 43 saved feasible vectors satisfy the 10^{-5} threshold; the largest violation is 9.90×10^{-7}. The two Big-M runs without accepted feasible solutions remain in the 45-run denominator. This numerical check establishes neither exact feasibility nor global optimality.

Eight of the 21 discrepancies are only 1.09–3.54 times the tolerance and are sensitive to numerical feasibility conventions. These conflicting bounds and `optimal` reports provide no independent global certificate or intrinsic method ranking; their cause in GDPopt or its subsolvers remains unresolved.

D: something is wrong here, it appears as if there were two table captions A: Shortened the caption and moved the explanation below the table, with this review comment outside the caption.

Table S7: Reported lower bounds exceeding checked feasible objectives ($\rho > 1$).

ID	Model	Method	Reported L	$v_{\max}$	C	ρ
S01	`CSTR5`	RIC	3.06201	2.60e-11	C01	1.29†
S02	`CSTR5`	GLOA	3.13020	9.85e-12	C01	2.23e+3†
S03	`CSTR5`	LOA	3.06201	2.60e-11	C01	1.29†
S05	`CSTR5`	Hull	3.06201	9.34e-7	C01	1.09†
S06	`CSTR10`	RIC	2.96605	1.62e-10	C02	2.65e+3†
S07	`CSTR10`	GLOA	3.44796	3.11e-15	C02	1.93e+4†
S08	`CSTR10`	LOA	2.96605	1.62e-10	C02	2.65e+3†
S10	`CSTR10`	Hull	2.88951	9.37e-7	C02	3.54†
S11	`CSTR15`	RIC	2.82939	1.33e-15	C03	1.30†
S12	`CSTR15`	GLOA	3.56196	1.01e-15	C03	2.59e+4†
S13	`CSTR15`	LOA	2.82939	1.33e-15	C03	1.30†
S16	`CSTR20`	RIC	2.79941	1.58e-15	C04	1.67†
S17	`CSTR20`	GLOA	3.61612	4.60e-11	C04	2.92e+4†
S18	`CSTR20`	LOA	2.79941	1.58e-15	C04	1.67†
S21	`CSTR25`	RIC	2.79125	1.38e-10	C05	3.53e+2†
S22	`CSTR25`	GLOA	3.64786	2.22e-14	C05	3.12e+4†
S27	`CSTR30`	GLOA	3.66872	7.11e-15	C06	3.25e+4†
S32	`Column`	GLOA	20355.1	1.15e-13	C07	4.65e+3†
S35	`Column`	Hull	20044.2	1.00e-8	C07	3.06e+3†
S42	`Dynamic`	GLOA	-197.140	9.85e-12	C09	7.01e+3†
S45	`Dynamic`	Hull	-196.931	1.00e-7	C09	7.11e+3†

C identifies the comparison solution in Table S8. Displayed values are rounded; flags use unrounded values.

Table S8: Checked saved comparison solutions; displayed values are rounded.

ID	Model	Method	Recorded f_w	$v_{\max}$
C01	`CSTR5`	Big-M	3.06198	9.40e-7
C02	`CSTR10`	Big-M	2.88941	9.76e-7
C03	`CSTR15`	Hull	2.82936	9.47e-7
C04	`CSTR20`	Hull	2.79936	9.90e-7
C05	`CSTR25`	Hull	2.78143	9.90e-7
C06	`CSTR30`	Big-M	2.76957	9.68e-9
C07	`Column`	RIC	19449.9	1.01e-13
C08	`Small Batch`	Big-M	167428	9.00e-7
C09	`Dynamic`	Big-M	-212.009	1.00e-7

Additional tightened-feasibility check for `CSTR15`. An `IPOPT` 3.14.19 solve retained the archived LOA witness W03's discrete configuration and starting values and targeted an original-GDP residual of 10^{-8} (SECQUOIA, 2026a). W03 is distinct from the current `SCIP` comparison solution C03. Only exactly satisfied constant rows were omitted from the computational clone; the independent check retained all original constraints. The returned objective was 2.82939, with maximum unscaled violation 1.11×10^{-16}. The new GLOA lower bound exceeds this objective by 0.732565, or approximately 2.59×10^4 times the audit tolerance. The discrepancy persists under the tighter check, but its cause remains unresolved.

Additional formulation check for `Dynamic` Hull. The saved Big-M point has objective -212.009, approximately 15.0776 below the Hull lower bound -196.931. It satisfies both the original GDP and a reconstructed Hull formulation at 10^{-5}, with maximum violation approximately 10^{-7}. The cause of the

discrepancy remains unresolved. D: This is more of an implementation detail, why including it? A: Removed the export and implementation details, retaining only the feasibility check relevant to the reported bound.

S9 Additional candidate-search and sampling comparisons

Figure S1 shows when successful process runs first reach the target.

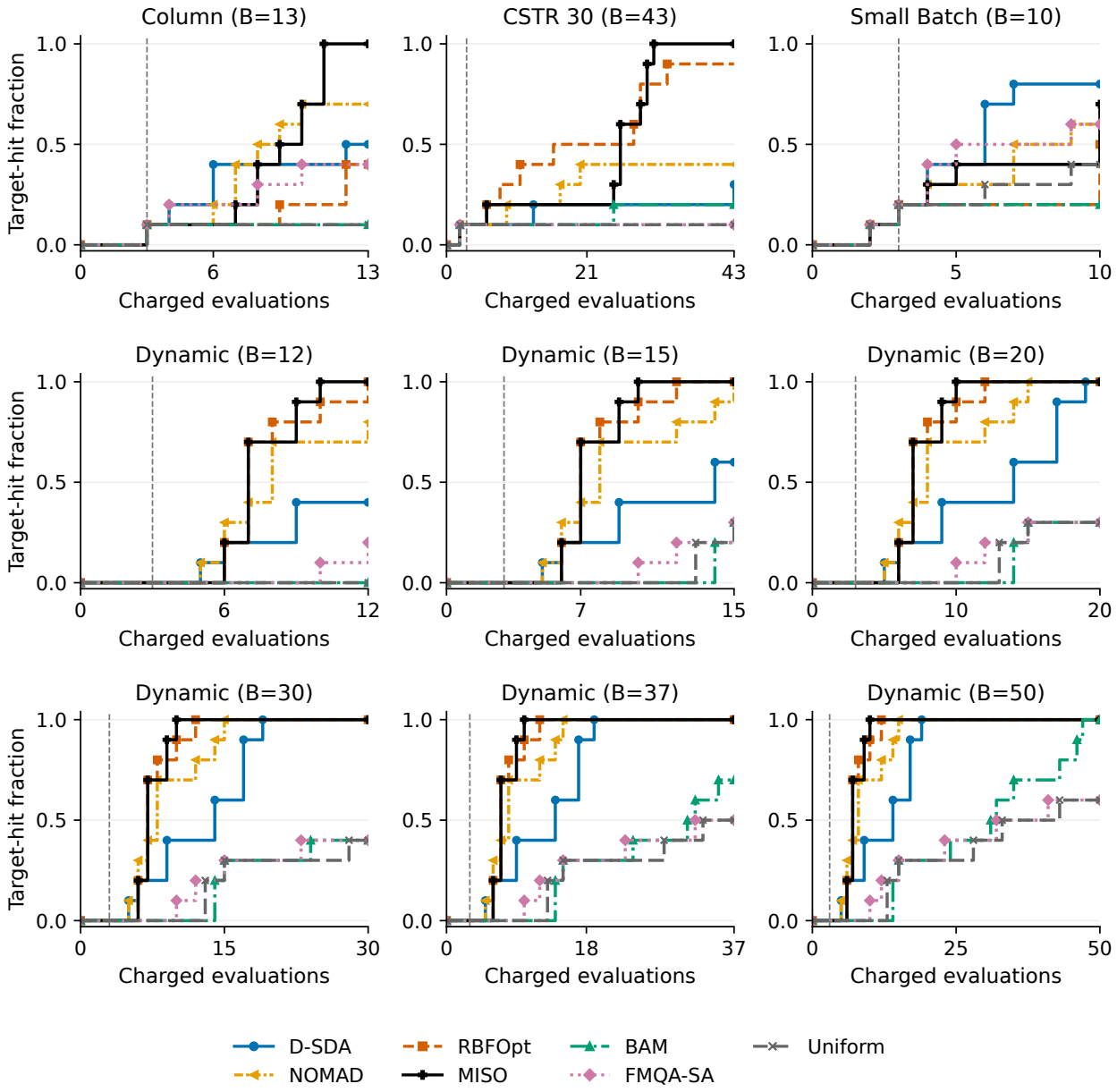


Figure S1: Cumulative target attainment versus charged evaluations for nine illustrative process conditions, using the success definition of the main paper. All ten seeds remain in every denominator, and initialization, unaccepted evaluations, and repeated configurations consume the budget. Steps mark the first accepted target observation on successful runs. The gray vertical line at three evaluations marks the end of the shared initialization. Line styles and markers distinguish methods. All 14 conditions appear in Table 2 of the main paper.

S9.1 Paired differences and sampling controls

The paired comparisons quantify differences in target-hit rates under shared initial designs, matching case, budget, and seed. Positive differences favor the first method and are expressed in percentage points. For Grid, the descriptive percentile bootstrap resamples the 21 intact function vectors, each containing ten paired seed differences, rather than treating 210 observations as exchangeable Bernoulli draws. For Process, paired seeds are resampled within each of the 14 conditions; the six related `Dynamic` budgets are not pooled for inference. The seed is 20261020 with 2,000 bootstrap replications and deterministic case/seed ordering. Intervals are exploratory and unadjusted for multiple comparisons; ten-seed boundary cases may yield degenerate intervals, and zero inclusion does not establish equivalence.

An interval entirely above zero favors the first method under this resampling; an interval spanning zero leaves the direction uncertain. There is no preset cutoff for practical importance. For example, `BAM` matches its prefix control at $B = 10$ but exceeds it by 10.95 percentage points at $B = 100$, so initial sampling alone does not account for its final target-hit rate. D: SO what does this mean? WHy are you reporting this? If the difference is greater than a value does it mean something? A: Explained the sign and interval interpretation and used the BAM-prefix comparison to show why the paired results are reported.

Table S9: Selected paired success differences, in percentage points. The first method minus the second defines the sign; Process intervals concern the stated condition only.

Grid

B	Contrast	Difference	95% interval
10	`RBFOpt` – Uniform	13.33	[5.71, 24.29]
100	`RBFOpt` – Uniform	60.00	[42.86, 75.71]
10	`BAM` – `BAM`-prefix control	0.00	[0.00, 0.00]
100	`BAM` – `BAM`-prefix control	10.95	[0.48, 23.81]
10	`FMQA-SA` – Uniform	0.95	[-0.95, 2.86]
100	`FMQA-SA` – Uniform	40.00	[28.57, 52.38]
10	`D-SDA` – `RBFOpt`	-13.81	[-24.29, -6.19]
100	`D-SDA` – `RBFOpt`	-33.33	[-48.10, -19.52]

Process: ten paired seeds per condition

Case	B	Contrast	Outcomes	Difference	95% interval
`Dynamic`	12	`RBFOpt` – `D-SDA`	4/6/0/0	60	[30, 90]
`Small Batch`	10	`FMQA-SA` – Uniform	2/4/2/2	20	[-30, 60]

Outcomes list both successful / first only / second only / neither successful.

S9.2 Exact paired tests for process conditions

Table S10 lists the seed-paired target outcomes of `RBFOpt` and `D-SDA` in every process condition and of `BAM` and `RBFOpt` on `CSTR5`. The p-values come from the two-sided exact McNemar test, which uses only the pairs in which exactly one method reaches the target. They are descriptive and not adjusted for multiple comparisons. In the 60 paired `CSTR` runs, `RBFOpt` reaches the target in 30 of the 31 discordant pairs.

Table S10: Seed-paired target outcomes and exact McNemar p-values. First only and second only count seeds in which only the first or the second method reaches the target.

Contrast	Case	B	Both	First only	Second only	Neither	p
RBFOpt vs D-SDA	`CSTR5`	8	2	6	0	2	0.0312
RBFOpt vs D-SDA	`CSTR10`	15	2	4	1	3	0.375
RBFOpt vs D-SDA	`CSTR15`	22	2	5	0	3	0.0625
RBFOpt vs D-SDA	`CSTR20`	29	3	4	0	3	0.125
RBFOpt vs D-SDA	`CSTR25`	36	3	5	0	2	0.0625
RBFOpt vs D-SDA	`CSTR30`	43	3	6	0	1	0.0312
RBFOpt vs D-SDA	`CSTR`, all sizes	–	15	30	1	14	2.98×10^{-8}
RBFOpt vs D-SDA	`Column`	13	3	1	2	4	1
RBFOpt vs D-SDA	`Small Batch`	10	4	1	4	1	0.375
RBFOpt vs D-SDA	`Dynamic`	12	4	6	0	0	0.0312
RBFOpt vs D-SDA	`Dynamic`	15	6	4	0	0	0.125
RBFOpt vs D-SDA	`Dynamic`	20	10	0	0	0	1
RBFOpt vs D-SDA	`Dynamic`	30	10	0	0	0	1
RBFOpt vs D-SDA	`Dynamic`	37	10	0	0	0	1
RBFOpt vs D-SDA	`Dynamic`	50	10	0	0	0	1
BAM vs RBFOpt	`CSTR5`	8	8	2	0	0	0.500

S9.3 Grid suite exclusions

Excluding Xin-She Yang 3 and Bohachevsky 2 preserves the main descriptive pattern. For the sampled Bohachevsky 2 lattice, $\cos(3\pi a_i) = 1$ at every coordinate, leaving an objective nearly redundant with sampled Sum of Squares after normalization. Without both functions, $B = 100$ rates are 84.7% for `MISO`, 79.5% for `RBFOpt`, 66.8% for `BAM`, 57.9% for `FMQA-SA`, 55.8% for `NOMAD`, 54.7% for the `BAM`-prefix control, 42.6% for `D-SDA`, and 17.9% for uniform. At $B = 10$, `BAM` and its prefix control each retain 47.9%, compared with 15.8% for `RBFOpt`, 14.7% for `MISO`, and 12.6% for `NOMAD`.

D: This section is irrelevant, if the QUBOs were not saved, then just repeat the whole experiment and have it cleanly worked out A: Removed the reconstructed FMQA diagnostics and their main-text reference; no experiment was rerun and no validation of unsaved historical weights or QUBOs is claimed.

S10 Seed spread, tolerance sensitivity, and search overhead

Table S11 compares exact target attainment with relaxed relative tolerances for the 980 primary process runs. The relative gap is $(f_{\text{best}} - f_{\text{ref}})/\max\{1, |f_{\text{ref}}|\}$, the normalization of the target rule in Equation (8). The exact column applies the target tolerance of 10^{-5}. Distinct finals is the largest number of different final objectives among the ten seeds of any condition in the family, and the miss-gap range covers every run that misses the exact target. Table S12 compares the median configuration-evaluation cost with the median search time per evaluation. Both are medians over runs of per-run averages, namely the summed point cost or the stored search time divided by the number of evaluations, over all seven methods for the evaluation cost. The 21 `NOMAD` Grid runs without a recorded search time are left out of its search median.

Every NLP subproblem solved by `GDPopt` in the process comparisons fixes a different configuration, as recorded in its execution logs before each subproblem. RIC and LOA solve one subproblem for each of the $N(N+1)/2$ logically admissible configurations of `CSTR5` to `CSTR20` and for each of the 37 admissible `Dynamic` configurations, and RIC also covers all 325 admissible configurations of `CSTR25`.

Table S11: Exact and relaxed target attainment and spread of final objectives across seeds. The relaxed columns count runs whose relative gap to f_{ref} is at most the stated value.

Family	Method	Exact	$\leq 0.1\%$	$\leq 1\%$	$\leq 5\%$	Distinct finals	Miss gap range (%)
	D-SDA	16/60	16	19	27	5	0.224–13.0
	NOMAD	21/60	21	21	27	3	2.23–13.0
	RBFOpt	45/60	45	45	47	3	2.23–12.5
`CSTR`	MISO	43/60	43	43	49	5	2.23–10.6
	BAM	18/60	18	60	60	3	0.224–0.692
	FMQA-SA	9/60	9	23	41	10	0.169–17.2
	Uniform	11/60	11	25	48	10	0.224–14.3
	D-SDA	5/10	5	8	10	4	0.536–1.14
	NOMAD	7/10	7	9	10	3	0.536–1.14
	RBFOpt	4/10	4	5	9	6	0.536–5.92
`Column`	MISO	10/10	10	10	10	1	–
	BAM	1/10	1	1	10	2	1.05–1.05
	FMQA-SA	4/10	4	4	10	4	1.05–2.76
	Uniform	1/10	1	2	9	8	0.536–5.22
	D-SDA	8/10	8	8	8	2	22.2–22.2
	NOMAD	6/10	6	6	6	2	6.64–6.64
	RBFOpt	5/10	5	5	5	3	6.64–22.2
`Small Batch`	MISO	7/10	7	7	7	2	6.64–6.64
	BAM	2/10	2	2	2	3	6.64–8.23
	FMQA-SA	6/10	6	6	6	5	6.64–22.2
	Uniform	4/10	4	4	4	4	6.64–22.2
	D-SDA	50/60	50	50	54	5	2.07–36.3
	NOMAD	58/60	58	58	60	3	2.07–2.66
	RBFOpt	60/60	60	60	60	1	–
`Dynamic`	MISO	60/60	60	60	60	1	–
	BAM	27/60	27	27	46	4	2.07–7.01
	FMQA-SA	23/60	23	23	42	7	2.07–36.3
	Uniform	21/60	21	21	38	6	2.07–30.7

Table S12: Median configuration-evaluation cost and median search time per evaluation.

		Search time per evaluation (ms)						
Family	Evaluation cost (s)	D-SDA	NOMAD	RBFOpt	MISO	BAM	FMQA-SA	Uniform
Grid-GDP	0.390	0.0777	50.5	31.1	231	3.77	604	0.0336
`CSTR`	1.46	1.24	72.2	24.2	287	9.01	572	0.517
`Column`	2.50	0.941	3.25	17.4	51.7	1.36	527	0.551
`Small Batch`	0.422	1.02	4.75	17.0	66.8	1.69	487	0.536
`Dynamic`	0.836	0.893	96.9	98.4	1860	9.34	567	0.471

S11 NOMAD and MISO runs

The NOMAD and MISO runs use the stored responses, shared initial configurations, seeds, budgets, penalties, and reference targets of the other five methods. Neither method receives the target as a stopping criterion. They were executed after the other methods, on a shared server with six workers, each restricted to one logical CPU on a different physical core and to one OpenMP and BLAS thread; the cores were not reserved exclusively. The server used Python 3.11.16, PyNomadBBO 4.6.0, soogo 2.1.0, NumPy 2.4.6, and SciPy 1.17.1. As for the other methods, completion times add the measured search time of each run to the recorded cost of every evaluated configuration, so they do not depend on the speed of the server's NLP solves.

Termination. All 1,190 MISO runs use their full budget. NOMAD stops before the budget in 77 of the 140 process runs and in 697 of the 1,050 Grid runs, including all 210 runs at $B = 100$, where it stops after a median of 31.5 evaluations. Its Grid target fraction is therefore 60.0% at both $B = 70$ and $B = 100$.

NOMAD **crashes and reruns.** Of the 1,190 NOMAD runs, 1,046 completed on the first attempt. The other 144 ended with a segmentation fault after their last new evaluation. Of these, 110 completed on the first rerun and 13 on the second, each after evaluating the same configurations in the same order as the crashed attempt, and 21 crashed on all three attempts after identical evaluations. All 21 are Grid runs at $B = 100$, and 16 of them had reached the target before the crash. Scoring them on the evaluations completed before the crash gives NOMAD 526 of 1,050 Grid targets and 92 of 140 process targets; counting all 21 as misses gives 510 Grid targets, or 52.4% instead of 60.0% at $B = 100$. These 21 runs have no recorded search time, so the NOMAD timing summaries at Grid $B = 100$ use the other 189 runs. For a completed rerun, the search time is that of the completed attempt; the time spent in crashed attempts is not recorded. No MISO run failed.

Exploratory paired tests. Table S13 compares NOMAD and MISO with D-SDA, RBFOpt, and each other in every process condition, a family of 70 exact McNemar tests defined after the runs were completed. With a Holm correction over the 70 tests, the smallest adjusted p-value is 0.547, for MISO against NOMAD on CSTR20, so no contrast is significant at the 0.05 level. Ten seeds give each test little power, and a nonsignificant test does not establish equivalence. The six Dynamic budgets reuse the same seeds and are not independent replications. The last row pools the six CSTR sizes for the descriptive comparisons in the main text; it is not part of the corrected family.

Table S13: Seed-paired target outcomes of NOMAD and MISO. Each cell gives the number of seeds in which only the first or only the second method reaches the target, and the unadjusted two-sided exact McNemar p-value.

Case	B	NOMAD vs D-SDA	NOMAD vs RBFOpt	MISO vs D-SDA	MISO vs RBFOpt	MISO vs NOMAD
Column	13	3:1 0.625	4:1 0.375	5:0 0.0625	6:0 0.0312	3:0 0.250
CSTR5	8	2:0 0.500	1:5 0.219	2:0 0.500	0:4 0.125	1:1 1
CSTR10	15	0:0 1	1:4 0.375	0:1 1	0:4 0.125	0:1 1
CSTR15	22	0:0 1	0:5 0.0625	5:0 0.0625	1:1 1	5:0 0.0625
CSTR20	29	0:1 1	0:5 0.0625	7:0 0.0156	3:0 0.250	8:0 0.00781
CSTR25	36	4:1 0.375	2:4 0.688	7:0 0.0156	2:0 0.500	4:0 0.125
CSTR30	43	1:0 1	1:6 0.125	7:0 0.0156	1:0 1	6:0 0.0312
Small Batch	10	1:3 0.625	2:1 1	1:2 1	2:0 0.500	3:2 1
Dynamic	12	4:0 0.125	0:2 0.500	6:0 0.0312	0:0 1	2:0 0.500
Dynamic	15	4:0 0.125	0:0 1	4:0 0.125	0:0 1	0:0 1
Dynamic	20	0:0 1	0:0 1	0:0 1	0:0 1	0:0 1
Dynamic	30	0:0 1	0:0 1	0:0 1	0:0 1	0:0 1
Dynamic	37	0:0 1	0:0 1	0:0 1	0:0 1	0:0 1
Dynamic	50	0:0 1	0:0 1	0:0 1	0:0 1	0:0 1
CSTR, all sizes	–	7:2 0.180	5:29 3.86×10^{-5}	28:1 1.12×10^{-7}	7:9 0.804	24:2 1.05×10^{-5}

S12 Process D-SDA restarts

The process `D-SDA` of the main comparison descends once from the best of the three shared initial configurations (S). Variant R applies the Grid rule and descends from all three shared initial configurations, best first, while budget remains. Variant RR continues after R with descents from uniformly drawn unvisited configurations until the budget is exhausted, and it uses the full budget in all 140 runs. No variant receives the reference target. All variants read the stored responses and, like the main comparison, never evaluate a configuration twice. Table S14 shows that R and RR evaluate 1,055 and 1,249 more configurations than S and add one target, on seed 4 of `CSTR30`, so target attainment rises from 79 to 80 of the 140 runs. Both restart variants improve the final objective in the same 14 `CSTR` runs. In each of them, S ends at $(5, 1)$, 8.3 to 13.0% above the reference, and the restarts end at the target in the one new hit and 0.4 to 8.6% above it in the other 13. On `CSTR10` to `CSTR30`, 31 of the 36 single-descent misses end at $(5, 1)$.

Table S14: Process `D-SDA` with a single descent (S), restarts from the ranked shared initial configurations (R), and additional random restarts (RR). Completion times add the measured search time to the recorded configuration costs.

		Target hits			Full-budget runs		Median evaluations			Median completion (s)		
Case	B	S	R	RR	S	R	S	R	RR	S	R	RR
`Column`	13	5	5	5	9	10	13	13	13	35.44	35.69	35.69
`CSTR5`	8	2	2	2	8	10	8	8	8	10.67	10.71	10.71
`CSTR10`	15	3	3	3	3	10	11.5	15	15	17.61	21.57	21.57
`CSTR15`	22	2	2	2	2	10	15.5	22	22	25.68	35.19	35.19
`CSTR20`	29	3	3	3	0	10	19	29	29	32.31	47.49	47.49
`CSTR25`	36	3	3	3	2	9	23.5	36	36	45.49	60.78	60.78
`CSTR30`	43	3	4	4	1	8	24.5	43	43	47.17	78.88	78.88
`Small Batch`	10	8	8	8	10	10	10	10	10	4.23	4.23	4.23
`Dynamic`	12	4	4	4	9	10	12	12	12	12.22	12.25	12.25
`Dynamic`	15	6	6	6	6	10	15	15	15	15.32	15.76	15.76
`Dynamic`	20	10	10	10	3	10	18	20	20	20.02	20.76	20.76
`Dynamic`	30	10	10	10	0	10	18	30	30	20.02	27.13	27.13
`Dynamic`	37	10	10	10	0	2	18	34.5	37	20.02	28.01	30.05
`Dynamic`	50	10	10	10	0	0	18	34.5	50	20.02	28.01	37.00

S13 Joint comparison with conventional GDP methods

Figure S2 compares the eight process conditions not shown in the main joint completion-time figure. Each DFO condition has ten seeds, whereas each conventional GDP method has one run per problem. Timing and target criteria follow Figure 2 of the main paper. Table S15 reports all nine models, including final objectives and termination flags.

Table S15: Full-process-GDP results from one run per method and model. Entries report the final objective and measured full-run completion time (s) in parentheses, including model-template preparation; runs stopped by the 1,800 s time limit are shown as 1800+. `GDPopt` uses `SCIP` for master problems and `IPOPT` for NLP subproblems; Big-M and Hull retain their `SCIP` results. Bold marks the lowest objective in each row and ties within $10^{-5}\max\{1, |f_{\min}|\}$, where $f_{\min}$ is the lowest objective in that row. Feasibility means numerical feasibility under the original-GDP residual test at 10^{-5}, not exact feasibility.

Model	RIC	GLOA	LOA	Big-M	Hull
`CSTR5`	3.06201† (15.38)	3.13020†‡ (4.57)	3.06201† (24.58)	**3.06198** (53.68)	3.06201† (15.43)
`CSTR10`	2.96605†‡ (134.31)	3.44796†‡ (3.10)	2.96605†‡ (140.38)	**2.88941** (771.36)	2.88951† (71.63)
`CSTR15`	2.82939† (283.77)	3.56196†‡ (6.17)	2.82939† (445.43)	-^{TL} (1800+)	**2.82936** (284.63)
`CSTR20`	2.79941† (607.72)	3.61612†‡ (7.46)	2.79941† (733.53)	2.80413^{TL} (1800+)	**2.79936** (1150.67)
`CSTR25`	2.79125†‡ (1042.79)	3.64786†‡ (5.20)	2.79125^{TL} (1800+)	2.78771^{TL} (1800+)	**2.78143** (1701.75)
`CSTR30`	2.77162^{TL} (1800+)	3.66872†‡ (5.52)	2.87314^{TL} (1800+)	**2.76957**^{TL} (1800+)	**2.76957**^{TL} (1800+)
`Column`	**19449.9** (124.03)	20355.1†‡ (42.41)	**19449.9** (199.22)	-^{TL} (1800+)	22271.5$^{\mathrm{TL}\dagger}$ (1800+)
`Dynamic`	**-212.009** (149.39)	-197.140†‡ (238.96)	**-212.009** (550.49)	**-212.009**^{TL} (1800+)	-196.931†‡ (921.28)
`Small Batch`	**167428** (13.15)	**167428** (10.32)	**167428** (5.61)	**167428** (0.51)	**167428** (0.50)

TL: reported `maxTimeLimit` under the configured 1,800 s limit. Runs without TL report `optimal`.
Dash: no accepted feasible solution.
†: reported lower bound exceeds the best checked saved numerically feasible objective f by more than $10^{-5}\max\{1, |f|\}$.
‡: reference target missed despite reported `optimal` and a better checked numerically feasible solution. D: SOmetimes you call it target, sometimes lookup reference, unify this A: Used reference target for the success criterion; lookup now refers only to access to stored configuration responses.

Table S16: NLP workload in the process comparisons with `SCIP`–`IPOPT` GDPopt. DFO entries are median counts of corresponding fixed-configuration NLP evaluations over ten seeds; GDPopt entries count confirmed IPOPT NLP interface calls in one run. Logical and preprocessing rejections are excluded; rejected NLP returns are retained. These counts are not internal solver iterations or equally costly subproblems.

Case	B	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform	GLOA	LOA	RIC
CSTR5	8	6	6	6	5.5	4.5	4	15	15
CSTR10	15	10	12	8.5	9	7.5	2	55	55
CSTR15	22	17	16.5	11	15	11	4	120	120
CSTR20	29	21	20.5	17	17.5	15.5	4	210	210
CSTR25	36	25	25.5	18.5	23	21	2	315	325
CSTR30	43	26.5	31.5	21.5	30.5	21.5	2	243	455
Column	13	11	12	7.5	7	7	18	38	38
Small Batch	10	4.5	6	5	5	4	12	5	12
Dynamic	12	10	10	8	6.5	6	20	37	37
Dynamic	15	13	13	9	8	6.5	20	37	37
Dynamic	20	17.5	16	11	11	9.5	20	37	37
Dynamic	30	22	21	17	14.5	15	20	37	37
Dynamic	37	22	25	19	19	19	20	37	37
Dynamic	50	22	31	26	24.5	26	20	37	37

The same single Dynamic GDPopt execution is repeated across budget rows for reference. Internal NLP counts for direct Big-M and Hull are unavailable and are not treated as zero. Every `GDPopt` NLP call fixes a different configuration; `CSTR`N has $N(N+1)/2$ logically admissible configurations, and `Dynamic` has 37 (Section S10).

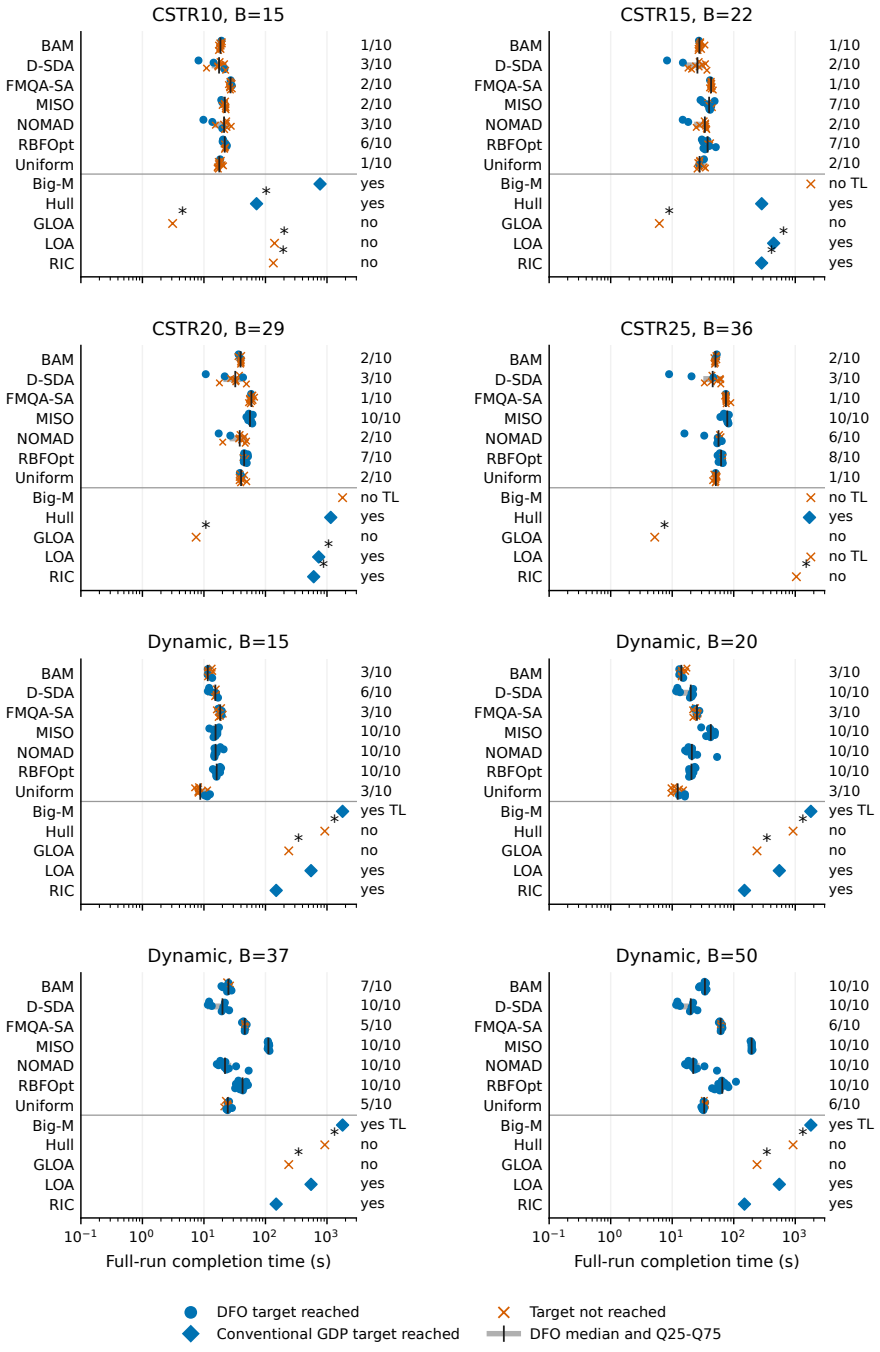


Figure S2: Joint full-run completion-time comparison for the eight additional process conditions. Dots show successful DFO seeds, diamonds successful single conventional runs, and crosses target misses. Gray segments and ticks show Q25–Q75 and medians across all ten DFO seeds. TL denotes time-limit termination; asterisks mark bound inconsistencies (Section S8). The Dynamic panels reuse the same conventional observations.